

计算机组成原理与系统结构

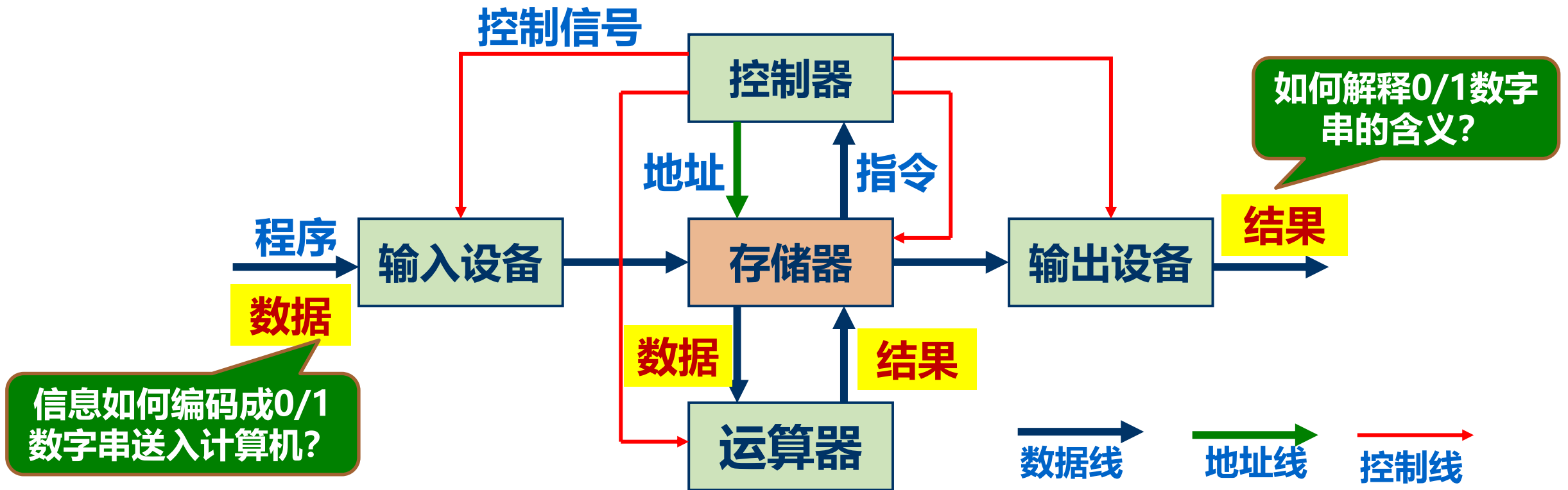
第3章 信息编码 与数据表示论





课程学习主线

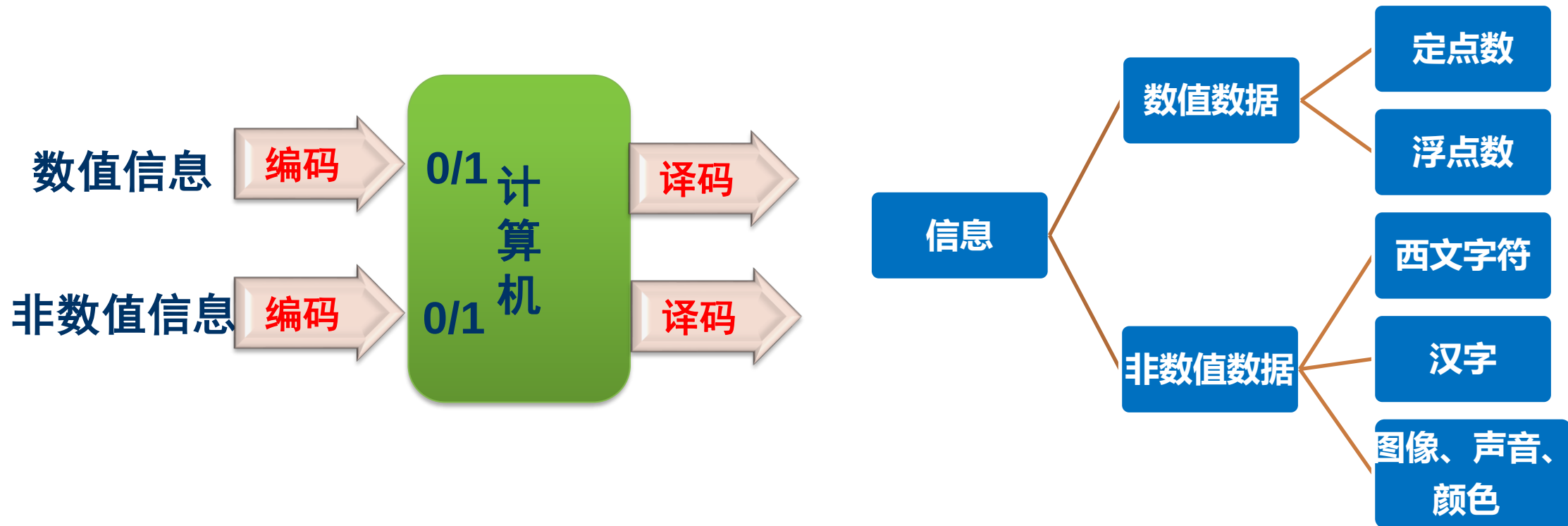
❖ 一张图：现代计算机结构框图（以存储器为中心）



❖ 一句话：计算机工作的过程就是循环往复地**取指令**、**分析指令**、**执行指令**的过程。



第3章 信息编码与数据表示





- ❖ 3.1 数制
- ❖ 3.2 数值数据的格式
- ❖ 3.3 定点机器数的表示方法
- ❖ 3.4 浮点机器数的表示方法
- ❖ 3.5 非数值数据的表示
- ❖ 3.6 校验码
- ❖ 3.7 现代计算机系统的数据类型
- ❖ 本章小结



3.1 数制



3.1.1 进位计数制



3.1.2 数制转换



3.1.3 十进制数的编码

本节复习+自学，课前小测





3.1.1 进位计数制

❖ 数制的两大要素:

- **基数R**: 允许使用的基本数码个数
- **权 W_i** : 权也称位权, 第*i*位上的数码的权重值, 即权与数码所处的位置*i*有关
- $W_i = R^i$

❖ 基数为R的数制称为**R进制数**: 逢R进1

❖ 何谓十进制、二进制、八进制、十六进制?

进制	基数R	权 W_i	数码符号
十进制	$R=10$	10^i	0~9
二进制	$R=2$	2^i	0、1
八进制	$R=8$	8^i	0~7
十六进制	$R=16$	16^i	0~9、A~F

❖ 假设任意数值N用R进制数来表示:

$$N = (D_{m-1}D_{m-2}\dots D_0 \cdot D_{-1}D_{-2}\dots D_{-k})_R$$

- D_i : 基本符号, $D_i \in [0, R-1]$, $i = -k, -k+1, \dots, m-1$; 小数点在 D_0 和 D_{-1} 之间

❖ 数值N的实际值为: 加权求和

$$N = \sum_{i=-k}^{m-1} (D_i \times R^i)$$

十进制

$$1635.458 = 1 \times 10^3 + 6 \times 10^2 + 3 \times 10^1 + 5 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} + 8 \times 10^{-3}$$

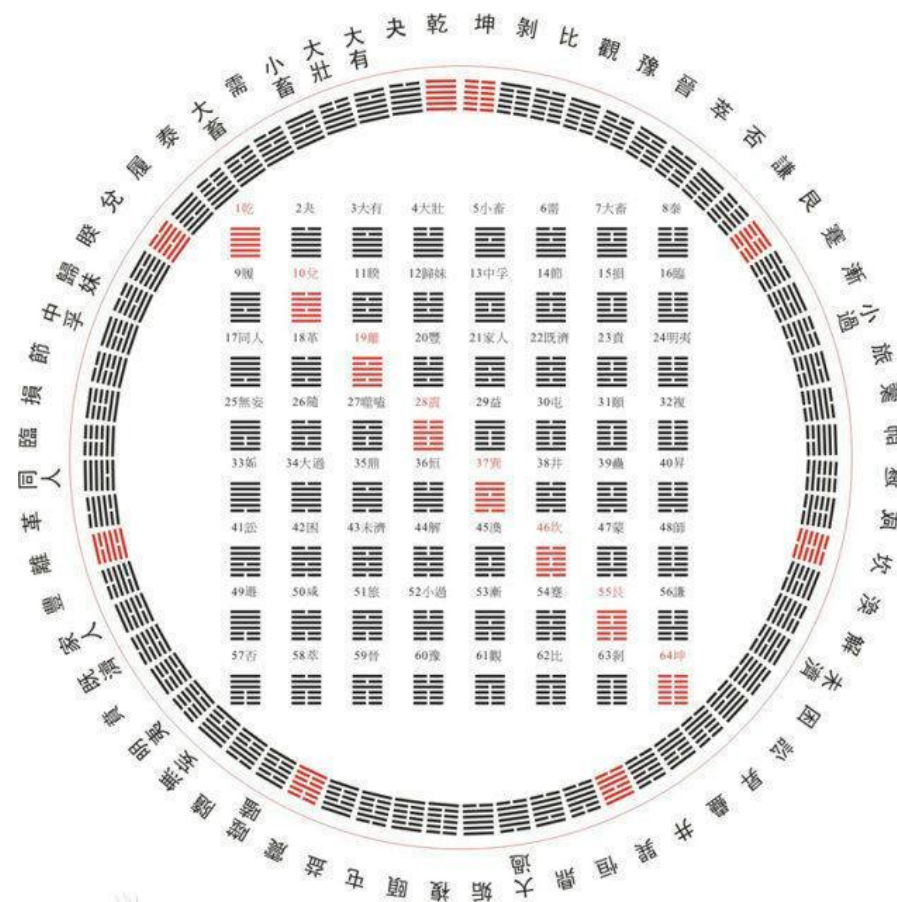


常用的几种数制的对应关系

十进制	二进制	八进制	十六进制	十进制	二进制	八进制	十六进制
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F
				16	10000	20	10

3.1.1 进位计数制

- ❖ 德国数学家莱布尼茨：发表论文首次提出二进制及其算术方法
 - 他承认中国古代的阴阳八卦与二进制记数有共同之处，甚至说自己是“重新发现了二进制”
- ❖ 《易经》：早在四千年前，就提出“太极生两仪，两仪生四象，四象生八卦”
 - “两仪”：指“阴爻”和“阳爻”，分别用“--”和“—”来标记
 - 如果将它们视为“0”和“1”，伏羲六十四卦方位图，就对应了6位二进制构成的数字0~63





3.1.1 进位计数制

❖ 计算机中为什么采用二进制表示数据？

- ① 具有二值状态的物理器件容易实现。
- ② 二进制数据的抗干扰性强，可靠性高。
- ③ 二进制的运算规则简单，硬件实现容易。
- ④ 具有逻辑特性，可代表“真假”、“是非”。





3.1.2 数制转换

- ❖ 1. 二进制数转换成十进制数
- ❖ 2. 十进制数转换成二进制数
- ❖ 3. 二进制数与八进制、十六进制数之间的转换





1. 二进制数转换成十进制数

❖ 转换方法：加权求和

$$N = \sum_{i=-k}^{m-1} (D_i \times R^i)$$

❖ 例3.1 将下列各数转化为十进制数。

- $(5AC.E6)_{16} = 5 \times 16^2 + 10 \times 16^1 + 12 \times 16^0 + 14 \times 16^{-1} + 6 \times 16^{-2} = (1452.8984375)_{10}$
- $(11011.011)_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = (27.375)_{10}$
- $(123.67)_8 = 1 \times 8^2 + 2 \times 8^1 + 3 \times 8^0 + 6 \times 8^{-1} + 7 \times 8^{-2} = (83.859375)_{10}$



1. 二进制数转换成十进制数

❖ 如何区分数字串是几进制的?

❖ 十进制 (Decimal)

❖ D

❖ 二进制 (Binary)

❖ B

❖ 八进制 (Octal)

❖ Q

❖ 十六进制 (Hexdecimal)

❖ H

❖ $(1011)_2 \rightarrow (1011)_B \rightarrow 1011B \rightarrow 1011b$

❖ $(123.45)_{10} \rightarrow (123.45)_D \rightarrow 123.45D \rightarrow 123.45$

❖ $(2B.D)_{16} = (2B.D)_H = (43.8125)_{10} = (53.64)_Q$





2. 十进制数转换成二进制数

❖ 转换方法

- **整数部分**：除以2取余，先得低位，直到商为0
- **小数部分**：乘2取整，先得高位，直到积为0或者达到精度要求为止

❖ 并不是所有的十进制小数都可以完全等价地转化为二进制数

❖ 当小数部分不能整除为二进制时，则乘以2取整的过程中，积不会为0；或者当小数部分转化为二进制位数很长，这时由精度来决定二进制位数。

- $(114.35)_{10} = (?)_2$ 无法整除
- $(0.6875)_{10} = (?)_2$ 位数太长



小数部分的精度要求

❖ 例3.2 将 $(114.35)_{10}$ 转化为二进制数，要求精度大于10%。

整数部分

2	114	余数	
2	57	0	(最低位)
2	28	1	
2	14	0	
2	7	0	
2	3	1	
2	1	1	
2	0	1	(最高位)

小数部分：取4位

	0.35	取整数
×	2	
	0.70	0 (最高位)
×	2	
	1.40	1
×	2	
	0.80	0
×	2	
	1.60	1 (最低位)

❖ 思考：若要求转换精度大于1%，则二进制小数取几位？

❖ 精度大于10%：要求 “=” 左右两边的十进制值的差的绝对值 < 10%

- $10\% > 2^{-4}$ ：只需取4位二进制小数即可满足要求





3. 二进制数与八进制、十六进制数之间的转换

❖ 二进制→八进制

- 以小数点为中心分别向两边分组，每三位一组，写出对应的八进制数字（不够位数则在两边加0补足3位）

❖ 二进制→十六进制

- 以小数点为中心分别向两边分组，每四位一组，写出对应的十六进制符号（不够位数则在两边加0补足4位）

❖ 八进制→二进制

- 将每位八进制数展开为3位二进制数，整数的最高位0和小数的最低位0可以略去

❖ 十六进制→二进制

- 将每位十六进制数展开为4位二进制数，整数的最高位0和小数的最低位0可以略去



3. 二进制数与八进制、十六进制数之间的转换

- ❖ 例3.3 将 $(11011.11001)_2$ 转化为八进制数、十六进制数和十进制数，将 $(571.23)_8$ 和 $(A8.E9)_{16}$ 转化为二进制数。

011 011. 110 010

0001 1011.1100 1000

- ❖ $(11011.11001)_2 = (33.62)_8 = (1B.C8)_{16} = (27.78125)_{10}$

- ❖ 也可以写成: $11011.11001B = 33.62Q = 1B.C8H = 27.78125D$

- ❖ $(571.23)_8 = (\underline{101} \ \underline{111} \ \underline{001}.\underline{010} \ \underline{011})_2$

- ❖ $(A8.E9)_{16} = (\underline{1010} \ \underline{1000}.\underline{1110} \ \underline{1001})_2$





3.1.3 十进制数的编码

❖ 如何在计算机内使用二进制来表示十进制数据？

❖ 1、二 - 十进制码 (BCD码)

❖ 2、十进制数串表示方法





1、二 - 十进制码 (BCD码)

❖ BCD (Binary Coded Decimal) 码:

- 使用二进制来编码十进制数字0 ~ 9

❖ 编码方法:

- 一般使用4位二进制编码来表示1位十进制数字
- 在16个编码中选用10个来表示数字0 ~ 9
- 不同的选择构成不同的BCD码

❖ 分类:

- **有权码**: 编码的每一位都有固定的权值, 加权求和的值即是表示的十进制数字。如8421码、2421码、5211码、4311码、84 -2-1码等
- **无权码**: 编码的每一位并没有固定的权, 主要包括格雷码、余3码等



1、二 - 十进制码 (BCD码)

十进制数	8421码	2421码	5211码	4311码	84-2-1码	格雷码	余3码
0	0000	0000	0000	0000	0000	0000	0011
1	0001	0001	0001	0001	0111	0001	0100
2	0010	0010	0011	0011	0110	0011	0101
3	0011	0011	0101	0100	0101	0010	0110
4	0100	0100	0111	1000	0100	0110	0111
5	0101	1011	1000	0111	1011	1110	1000
6	0110	1100	1010	1011	1010	1010	1001
7	0111	1101	1100	1100	1001	1000	1010
8	1000	1110	1110	1110	1000	1100	1011
9	1001	1111	1111	1111	1111	0100	1100



几种常见的BCD码

❖ ①8421码：最常用的一种BCD码

- 4位二进制数位的权从高到低依次是8、4、2、1
- 实际上就是十进制数字0 ~ 9的二进制编码本身
- 没有特别指出的情况下，BCD码通常就是指8421码

❖ ②余3码：对应的8421码+0011b

- 自补码：即任何两个相加之和等于9的编码，互为反码

❖ ③格雷码：

- 特点：又叫循环码，它的任何相邻的两个编码（例如2和3、7和8、9和0等）之间只有一位二进制位不同。
- 优点：是用它构成计数器时，在从一个编码变到下一个编码时，只有一个触发器翻转即可，波形更完美、可靠。格雷码的编码方案有许多种。

❖ 思考题：写出另一种十进制（0-9）的格雷码





2、十进制数串的代表方法

- ❖ **字符串形式**：用ASCII码来表示十进制数字或符号位，即1个字节存放1位十进制数字或符号位。
- ❖ **压缩的十进制数串形式**：用BCD码来表示十进制数字，即1个字节存放2个十进制的数字；符号位放在最低位数字位之后，一般用C（12）表示正号，用D（13）表示负号。
 - 例如 + 258被表示成258CH，占用两个字节，-34被表示为034DH，也占用两个字节。
- ❖ **共同点**：必须给出它的主存中的首地址和位长。
- ❖ **采用十进制表示数据的优点是**：
 - 对于需要大量地进行输入输出数据而运算简单的场合，大大减少了二-十进制转换，提高了机器的运行效率；
 - 十进制数串的位长可变，许多机器中规定该长度从0到31，有的甚至更长。不受定点数和浮点数统一格式的约束，从而提高了数据的表示范围和运算精度。





3.2 数值数据的格式

- ❖ **数值数据**：用于表示数的大小，其值确定，可与数轴上确切的点对应
- ❖ **非数值数据**：没有值大小的概念，表示字符、汉字、图像、声音等信息
- ❖ **计算机中如何表示数值数据？** 数值数据的格式要素，必须要考虑三点
 - **第一，符号如何表示？**
 - **第二，小数点如何表示？**
 - **第三，二进制数字如何排列来表示某个数值？**
- ❖ **机器数**：数值数据在计算机中的二进制表示形式
 - 机器数的**符号位**必须被数值化为二进制**0和1**
 - 机器数的**小数点**用**隐含规定**的方式来表达
 - 表示的**数值范围**受计算机字长的**限制**
 - **编码方法**：不同的排列方式→**原码、反码、补码、移码**

- 
- ❖ **真值**：机器数真正表示的数值数据
- 书写：**+/- 符号 数值**
(二进制或十进制)
 - **+178.59**
 - **-10111.1011b**



3.2 数值数据的格式

❖ 3.2.1 无符号数和有符号数

❖ 3.2.2 定点和浮点表示





3.2.1 无符号数和有符号数

- ❖ 计算机中参与运算的数值数据有两种：
 - **无符号数据 (Unsigned)** : 所有的二进制数据位数均用来表示数值本身, 没有正负之分
 - **带符号数据 (Signed)** : 则其二进制数据位, 包括符号位和数值位
- ❖ 当机器字长相同时, 无符号数和有符号数的表示范围是不同的
- ❖ 例如: 机器字长为16位
 - 无符号整数的表示范围: $0 \sim 65535$
 - 有符号补码表示的整数范围: $-32768 \sim +32767$
- ❖ 思考: 计算机硬件如何区分无符号数据和带符号数据呢?



3.2.1 无符号数和有符号数

❖ 例： Intel X86系列CPU

❖ 程序A：

- **JL**：将AX和BX当作有符号数据
- $AX = (-1)_{10}$, $BX = 1$
- 执行结果：转移到L1标号处执行

❖ 程序B：

- **JB**：将AX和BX当作无符号数据
- $AX = (65535)_{10}$, $BX = 1$
- 执行结果：不转移，顺序执行

```
MOV    AX,0FFFFH    ; AX=0FFFFH
MOV    BX,1          ; BX=0001H
CMP    AX,BX         ;结果影响标志位
JL    L1            ;有符号数小于转移
```

L1:

```
MOV    AX,0FFFFH    ; AX=0FFFFH
MOV    BX,1          ; BX=0001H
CMP    AX,BX         ;结果影响标志位
JB    L1            ;无符号数小于转移
```

L1:

❖ 结论：计算机硬件无需区分，但在运算或者执行比较跳转操作时，通过程序中不同的指令来区分



3.2.1 无符号数和有符号数

❖ 几乎所有CPU的指令系统：都区分有符号数和无符号数的指令

有符号运算指令

- X86: `IMUL BX`
- MIPS32: `addi t1,t3,0x8000`
- RISC-V: `slt x2,x5,x6`

无符号运算指令

- `MUL BX`
- `addiu t1,t3,0x8000`
- `sltu x2,x5,x6`

【-1】_补
=ffff ffff h
=4294967295

❖ `Printf( "%d\n" ,-1);`

- 显示结果：-1
- 因为d→显示带符号十进制数，将-1当做了带符号十进制数

❖ `Printf( "%u\n" ,-1);`

- 显示结果：4294967295
- 因为u→显示无符号十进制数，将-1当做了无符号十进制数



3.2.1 无符号数和有符号数

- ❖ **二进制位串：**本身并没有某种特定的含义，可以用来表示：
 - 有符号整数、无符号整数
 - 浮点数、字符串、机器指令.....
 - 究竟表示什么含义：取决于指令或者计算机对它们执行何种操作
- ❖ **程序处理的数据：**可能是正数或负数
 - 在某些情况下，只可能是正数（无符号数）：例如内存的地址，负的地址是无意义的
- ❖ **高级语言中：**通过数据类型来反映有符号和无符号数
- ❖ **C语言程序中如何进行有符号/无符号变量定义？**

❖ `int x`: 有符号数

❖ `unsigned int x`: 无符号整数





3.2.2 定点和浮点表示

❖ 计算机中：小数点的表示方法？

- 计算机中没有专用的部件用来表示小数点
- 机器数中，小数点及其位置是隐含规定的
- 有两种隐含方式：定点和浮点

❖ 1、定点表示法

❖ 2、浮点表示法

❖ 3、机器数的精度和表示范围

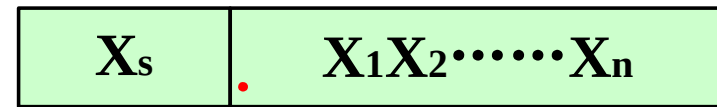




1、定点表示法

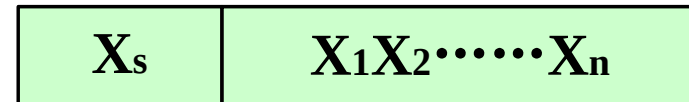
❖ **定点机器数 (Fixed Point)** : 小数点的位置是固定不变的, 两种位置:

■ **定点小数**: 纯小数, 隐含固定在符号位 X_s (最高位) 之后, 整数位是符号位



符号位 小数点隐含位置 数值位

■ **定点整数**: 纯整数, 小数点隐含固定在最低位之后, 最高位为符号位 X_s



符号位 数值位 小数点隐含位置

❖ **定点机器数的表示格式**: 符号 (sign) + 幅值 (magnitude)

❖ **定点机器数的编码方法**: 原码、补码、反码、移码

❖ **计算机中, 用定点整数格式表示整型数 int**

❖ **并无单纯的定点小数格式的数据类型**

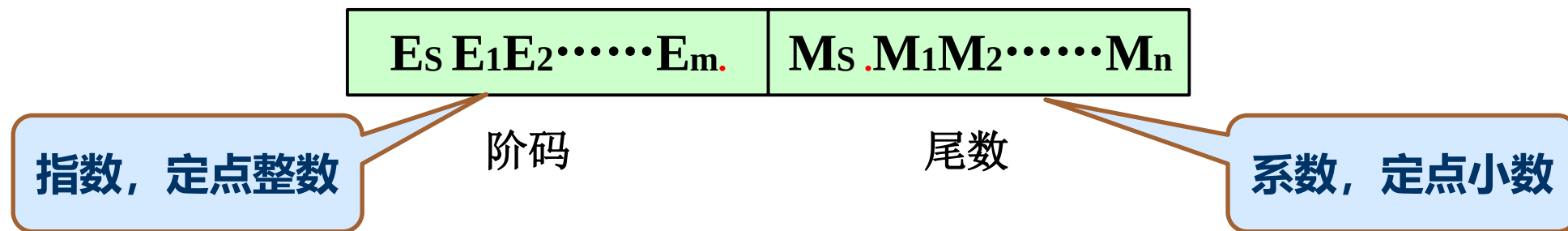




2、浮点表示法

❖ **浮点机器数**：将实数数值大小表示成科学计数法形式

$$N = M \times R^E$$



❖ 浮点数N由三部分构成：

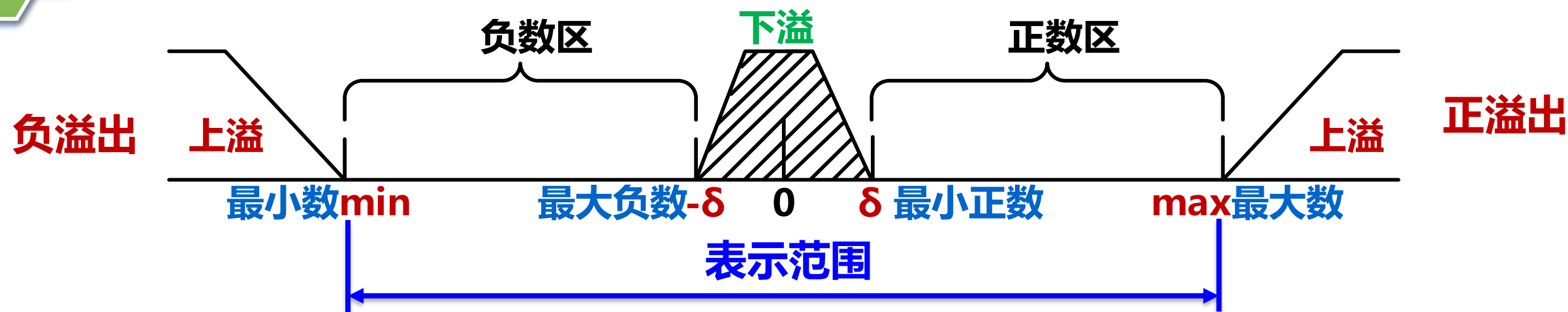
- **阶码的底R**：隐含表示，一般约定为2、8或16
- **尾数M**：系数，为定点小数
- **阶码E**：R的指数，为定点整数

❖ **数据真值的小数点位置是浮动的**：位置由阶码规定，不是尾数小数点的位置

- 阶码为正数：数据真正的小数点位置 在 尾数小数点的位置的左边 or 右边？
- 阶码为负数？



3、机器数的精度和表示范围



❖ **机器数的表示范围**：[最小数，最大负数]、0、[最小正数，最大数]

- 4个非0值，取决于机器数的编码方式、位数和格式

char x; 表示范围、精度?
8位补码整数?

❖ **上溢 (Overflow)**：溢出，报警

- 正溢出：> max
- 负溢出：< min

❖ **下溢 (Underflow)**：当做零处理

- $\in (-\delta, 0)$ $\in (0, \delta)$

❖ **精度/分辨率**：机器数所能表示的绝对值最小的非零值

- 最大负数：- δ (最靠近原点0)
- 最小正数：+ δ (最靠近原点0)



3、机器数的精度和表示范围

❖ **下溢**：位于最大负数和最小正数之间的数据（除0外），机器无法表示。

▪ **处理**：计算机直接将其**视为机器零**。

正溢出

❖ **上溢**：当一个数据大于最大（正）数，或者小于最小（负）数时，机器也无法表示，称为上溢，上溢又称**溢出**。

负溢出

▪ **处理**：计算机**置溢出标志位**，或者报警。

❖ **浮点数的机器零**：有两种情况

▪ （1）若浮点数的尾数为零，无论阶码为何值；

▪ （2）当阶码的值遇到比它能表示的最小值还要小时（阶码负溢出），无论其尾数为何值



3、机器数的精度和表示范围

- ❖ 在选择计算机的数值数据的表示方式时，需要考虑以下几个因素：
- ①要表示的数据类型（小数、整数、实数和复数）
 - ②可能要处理的数值范围
 - ③数值精确度
 - ④数据存储和处理所需要的硬件代价



定点数与浮点数比较

❖ 相同点:

- (1) 计算机所能表示的数据都是一系列离散的点
 - 问题: 存在于两个点之间的数据?
 - 解决: 计算机通常采用合适的舍入操作, 选取最近似的值来替代。
- (2) 计算机硬件的字长是有限的
 - 问题: 数据超出了机器数所能表示的最大界限?
 - 解决: 计算机必须能够产生“溢出”的异常报告。

❖ 不同点:

- 定点数可表示的点在数轴上是均匀的, 距离是等长的1 (定点整数) 或者 2^{-n} (定点小数)
- 浮点数则是分布不均匀、距离不相等的





3.3 定点机器数表示方法

❖ 3.3.1 原码表示法

❖ 3.3.2 补码表示法

❖ 3.3.3 反码表示法

❖ 3.3.4 移码表示法

❖ 四种机器数的对比

❖ 机器数→真值

❖ 从几个方面学习：

- 数学定义
- 表示方法
- 特点
 - 真值 “0” 的表示
 - 表示范围
 - 运算特点





3.3.1 原码表示法

❖ 1、数学定义：真值与编码之间的数学关系

X $[X]_{\text{原}}$

加权求和对应的
二进制(无符号数)

$$[+1101]_{\text{原}} = 0\ 1101$$

$$X = +1101 \longrightarrow +13$$

$$[X]_{\text{原}} = 0\ 1101 \longrightarrow 13$$

$$[-1101]_{\text{原}} = 1\ 1101$$

$$X = -1101 \longrightarrow -13$$

$$[X]_{\text{原}} = 1\ 1101 \longrightarrow 29$$

定点
整数

$$[X]_{\text{原}} = \begin{cases} X & X \geq 0 \\ 2^n - X & X \leq 0 \end{cases}$$

定点
小数

$$[X]_{\text{原}} = \begin{cases} X & X \geq 0 \\ 1 - X & X \leq 0 \end{cases}$$

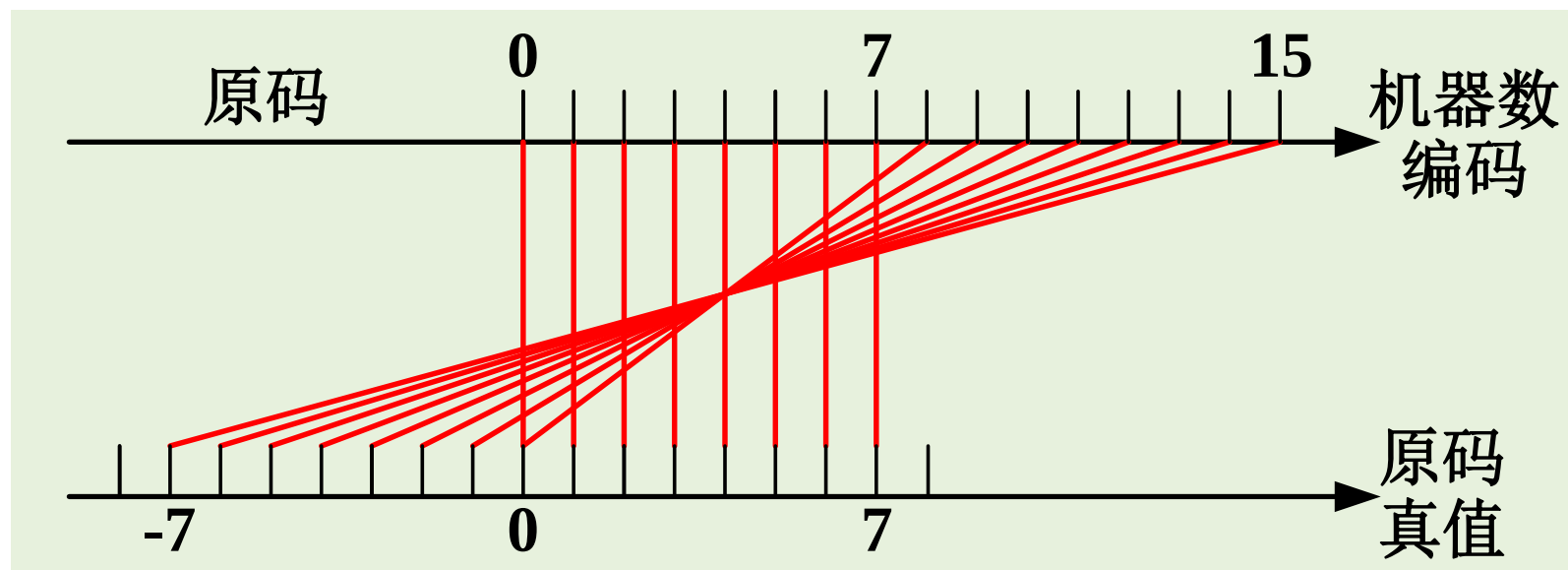


3.3.1 原码表示法

1、数学定义

❖ 4位原码机器数（整数）对应的真值

❖ 在数轴上的表示：编码与真值的对应



机器数	原码对应真值
0000	0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-0
1001	-1
1010	-2
1011	-3
1100	-4
1101	-5
1110	-6
1111	-7



3.3.1 原码表示法

❖ **2、表示方法：**最高位为符号位，其他位为数值位 **sign and magnitude 符号-幅值表示法**

- 符号位：0 - 正数，1 - 负数
- 数值位：与绝对值相同

❖ 对于定点整数：

- 若 $X = +X_1X_2\cdots X_n$ ，则 $[X]_{\text{原}} = 0, X_1X_2\cdots X_n$
- 若 $X = -X_1X_2\cdots X_n$ ，则 $[X]_{\text{原}} = 1, X_1X_2\cdots X_n$ 。

“,” 和 “.” 只用于助记，在计算机中并无专用部件来表示

❖ 对于定点小数：

- 若 $X = +0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{原}} = 0.X_1X_2\cdots X_n$
- 若 $X = -0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{原}} = 1.X_1X_2\cdots X_n$



❖ 例3.2 写出下面二进制数X的原码机器数，以及Y的8位原码机器数。

- $X=0.1001$ ■ $[X]_{\text{原}}=0.1001$
- $X=-0.1001$ ■ $[X]_{\text{原}}=1.1001$
- $X=1110$ ■ $[X]_{\text{原}}=0,1110$
- $X=-1110$ ■ $[X]_{\text{原}}=1,1110$

定点小数：整数位就是符号位

定点整数：最高位是符号位

■ $Y=-0.1001$ ，真值变成7位数值：

- $Y=-0.1001$ **000**
- $[Y]_{\text{原}}=1.1001000$

■ $Y=1110$ ，真值变成7位数值：

- $Y=\mathbf{000}1110$
- $[Y]_{\text{原}}=0, 0001110$



3.3.1 原码表示法

3、特点

❖ (1) 0的原码表示：有两种形式

■ +0：按**正数**来表示，符号位为0

• $[+0]_{\text{原}} = 00\dots0$

■ -0：按**负数**来表示，符号位为0

• $[-0]_{\text{原}} = 10\dots0$

❖ (2) 表示范围：对于n+1位原码机器数X，所能表示的数据范围为：

■ 定点整数： - $(2^n - 1) \leq X \leq 2^n - 1$

■ 定点小数： - $(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$

包括1位符号位，
n位数值位

■ 问题1：16位定点整数的原码表示范围？

■ 问题2：为什么有表示范围？

■ 问题3：为什么是这个表示范围？

• **最大数**二进制编码？对应的十进制数？

011.....111

• **最小数**二进制编码？对应的十进制数？

111.....111



❖ (3) 原码运算

- 使用原码进行加减乘除运算：规则类似于真值的加减乘除运算
- 符号位单独处理，不能参加运算，参加运算的是绝对值
- 加减运算：
 - 根据符号位决定进行的是绝对值的加还是减
 - 进行绝对值的加减运算
 - 根据绝对值的大小决定结果（和或者差）的符号
- 硬件设计与实现很困难

❖ 现代计算机中，原码主要用于表示浮点数的尾数



❖ 1、表示方法：最高位为符号位，其他位为数值位。

- 符号位：0 - 正数，1 - 负数。
- 数值位：与绝对值相同。

❖ 2、0的表示：不唯一

- $[+0]_{\text{原}} = 00\dots0$ $[-0]_{\text{原}} = 10\dots0$

❖ 3、表示范围：n + 1位原码机器数X，表示范围为：

- 定点整数：- $(2^n - 1) \leq X \leq 2^n - 1$
- 定点小数：- $(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$

❖ 4、数学表示：

定点
整数

$$[X]_{\text{原}} = \begin{cases} X & X \geq 0 \\ 2^n - X & X \leq 0 \end{cases}$$

定点
小数

$$[X]_{\text{原}} = \begin{cases} X & X \geq 0 \\ 1 - X & X \leq 0 \end{cases}$$





3.3.2 补码表示法

❖ 1、数学定义：

定点
整数

$$[X]_{\text{补}} = \begin{cases} X & X \geq 0 \\ 2^{n+1} + X & X < 0 \end{cases}$$

定点
小数

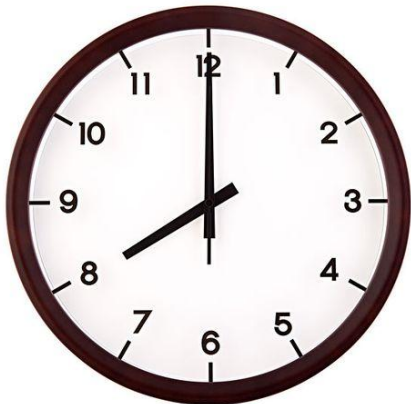
$$[X]_{\text{补}} = \begin{cases} X & X \geq 0 \\ 2 + X & X < 0 \end{cases}$$

2的补码: two' s complement

$$[X]_{\text{补}} = 2^{n+1} + X \pmod{2^{n+1}}$$

模2补码

$$[X]_{\text{补}} = 2 + X \pmod{2}$$



❖ 8点→6点

- 逆时针: -2
- 顺时针: +10

❖ 如何区分-2和+10?

- $(-2+24) \% 24 = 22$
- $(+10+24) \% 24 = 10$



3.3.2 补码表示法

1、数学定义

❖ 4位补码机器数（整数）对应的真值

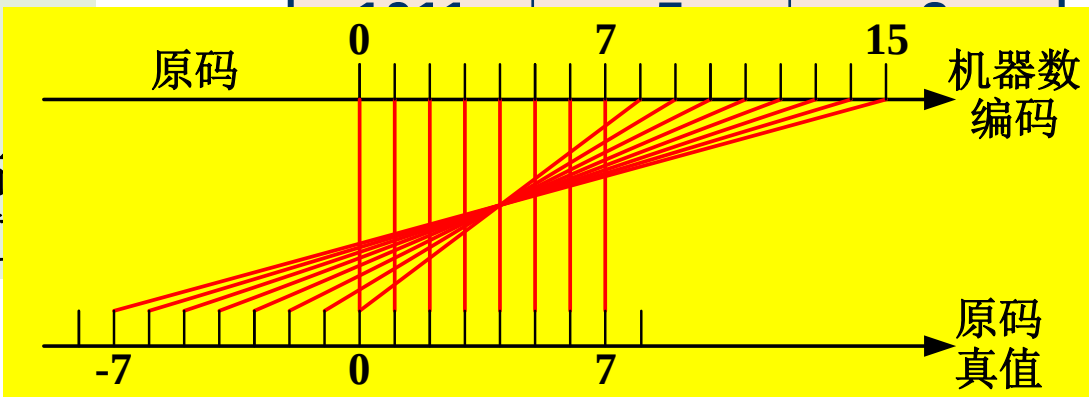
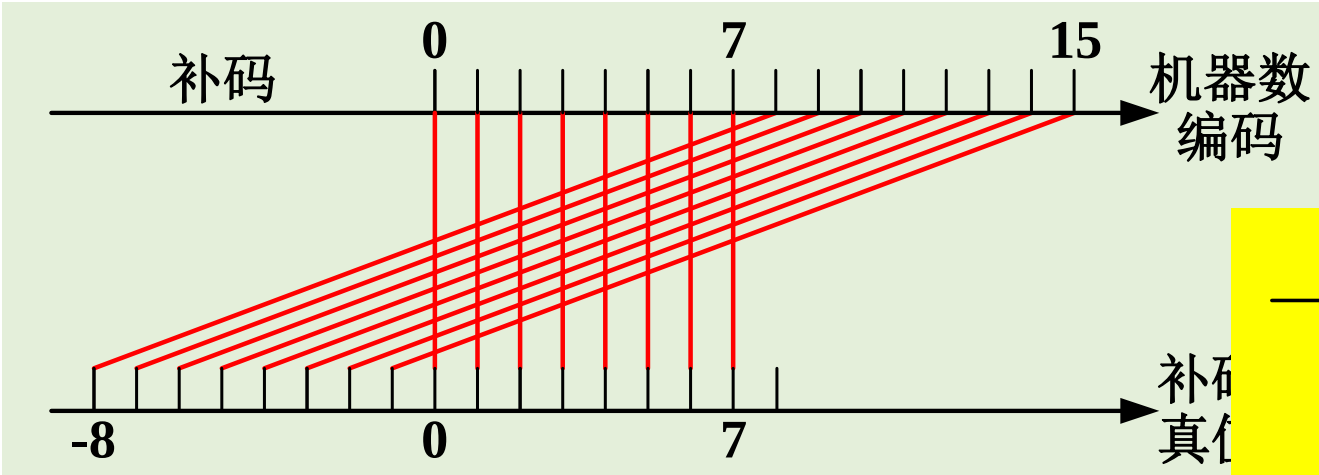
$[X]_{\text{补}} = \begin{matrix} X_0 & X_1 & X_2 & X_3 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ \text{权: } & -2^3 & 2^2 & 2^1 & 2^0 \end{matrix}$

$[X]_{\text{补}} = 1101 \quad X = ?$

$[X]_{\text{补}} = 0101 \quad X = ?$

真值=加权求和

❖ 在数轴上的表示：编码与真值的对应



机器数	补码真值	原码真值
0000	0	0
0001	+1	+1
0010	+2	+2
0011	+3	+3
0100	+4	+4
0101	+5	+5
0110	+6	+6
0111	+7	+7
1000	-8	-0
1001	-7	-1
1010	-6	-2
1011	-5	-3
1100	-4	-4
1101	-3	-5
1110	-2	-6
1111	-1	-7



3.3.2 补码表示法

❖ 2、表示方法：最高位为符号位，其他位为数值位。

- 符号位：0 - 正数，1 - 负数。
- 数值位：正数时，与绝对值相同；负数时，为绝对值取反后，末位加1。

❖ 对于定点整数：

- 若 $X = +X_1X_2\cdots X_n$ ，则 $[X]_{\text{补}} = 0, X_1X_2\cdots X_n$
- 若 $X = -X_1X_2\cdots X_n$ ，则 $[X]_{\text{补}} = 1, \overline{X_1} \overline{X_2} \cdots \overline{X_n} + 1$

❖ 对于定点小数：

- 若 $X = +0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{补}} = 0.X_1X_2\cdots X_n$
- 若 $X = -0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{补}} = 1.\overline{X_1} \overline{X_2} \cdots \overline{X_n} + 0.00\cdots 1$



❖ 例3.5 写出下面二进制数X的补码机器数，以及Y的8位补码机器数。

- $X = +0.1001$
- $X = -0.1001$
- $X = +1110$
- $X = -1110$
- $[X]_{\text{补}} = 0.1001$
- $[X]_{\text{补}} = 1.0110 + 0.0001 = 1.0111$
- $[X]_{\text{补}} = 0,1110$
- $[X]_{\text{补}} = 1,0001 + 1 = 1,0010$
- $Y = -0.1001$ ，真值变成7位数值：
 - $Y = -0.1001\underline{000}$
 - $[Y]_{\text{补}} = 1.0111000$
- $Y = 1110$ ，真值变成7位数值：
 - $Y = \underline{000}1110$
 - $[Y]_{\text{补}} = 0, 0001110$

思考：如果 $Y = -1110$ ，则其补码是什么？



3.3.2 补码表示法

3、特点

- ❖ (1) 0的表示: 0 的补码表示形式是唯一的, 按照正数和负数表示均为全零
 - $[+0]_{\text{补}} = 00\dots0$ $[-0]_{\text{补}} = 00\dots0$
- ❖ (2) 表示范围: 对于 $n+1$ 位补码机器数 X , 它所能表示的数据范围为:
 - 定点整数: $-2^n \leq X \leq 2^n - 1$ -2^n 的补码: $1,00\dots0$
 - 定点小数: $-1 \leq X \leq 1 - 2^{-n}$ -1 的补码: $1.00\dots0$
- ❖ (3) 补码运算
 - 补码的符号位同数值位一起参加运算
 - 可以将减法转化为加法
 - 简化了运算器的结构
- ❖ 计算机中的整型数据(int)均用补码来表示

- ❖ 32位微机中, C程序定义变量 x, y :
- ❖ `int x; unsigned int y;`
- ❖ 问: x, y 的数据取值范围?

- ❖ x 是32位补码表示的整数: $-2^{31} \leq X \leq 2^{31} - 1$
- ❖ y 是32位无符号整数: $0 \leq X \leq 2^{32} - 1$



❖ 1、表示方法：最高位为符号位，其他位为数值位。

- 符号位：0 - 正数，1 - 负数。
- 数值位：正数为绝对值；负数为绝对值取反，末位+1。

❖ 2、0的表示：唯一

- $[+0]_{\text{原}} = [-0]_{\text{原}} = 00\dots0$

❖ 3、表示范围：n + 1位补码机器数X，表示范围为：

- 定点整数： $-2^n \leq X \leq 2^n - 1$
- 定点小数： $-1 \leq X \leq 1 - 2^{-n}$
- 计算机中的整型数据 (int) 均用补码来表示。

❖ 4、数学表示：

$$[X]_{\text{补}} = 2^{n+1} + X \pmod{2^{n+1}}$$

$$[X]_{\text{补}} = 2 + X \pmod{2}$$





3.3.3 反码表示法

❖ 1、数学定义：

定点整数

$$[X]_{\text{反}} = \begin{cases} X & X \geq 0 \\ 2^{n+1} - 1 + X & X \leq 0 \end{cases}$$

1的补码：one's complement

定点小数

$$[X]_{\text{反}} = \begin{cases} X & X \geq 0 \\ 2 - 2^{-n} + X & X \leq 0 \end{cases}$$

❖ 2、表示方法：最高位为符号位，其他位为数值位。

- 符号位：0 - 正数，1 - 负数。
- 数值位：正数时，与绝对值相同；负数时，为绝对值取反

❖ 对于定点整数：

- 若 $X = +X_1X_2\cdots X_n$ ，则 $[X]_{\text{反}} = 0, X_1X_2\cdots X_n$
- 若 $X = -X_1X_2\cdots X_n$ ，则 $[X]_{\text{反}} = 1, \overline{X_1}\overline{X_2}\cdots\overline{X_n}$

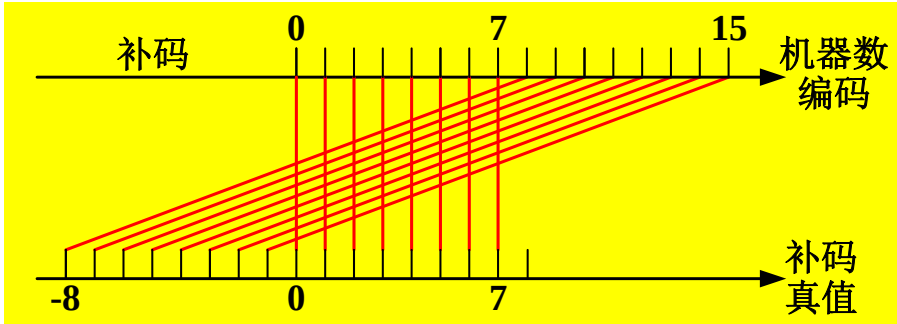
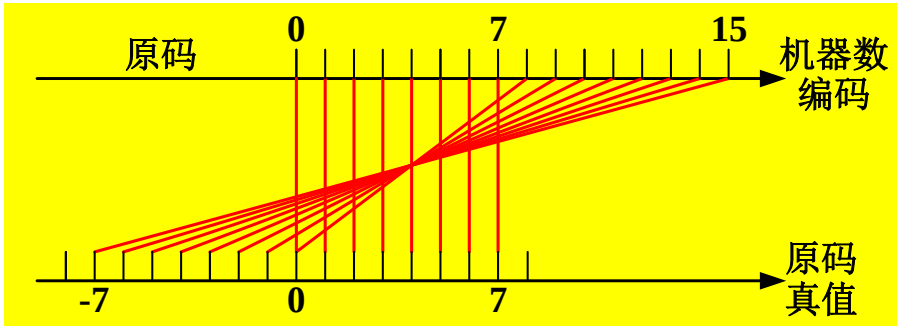
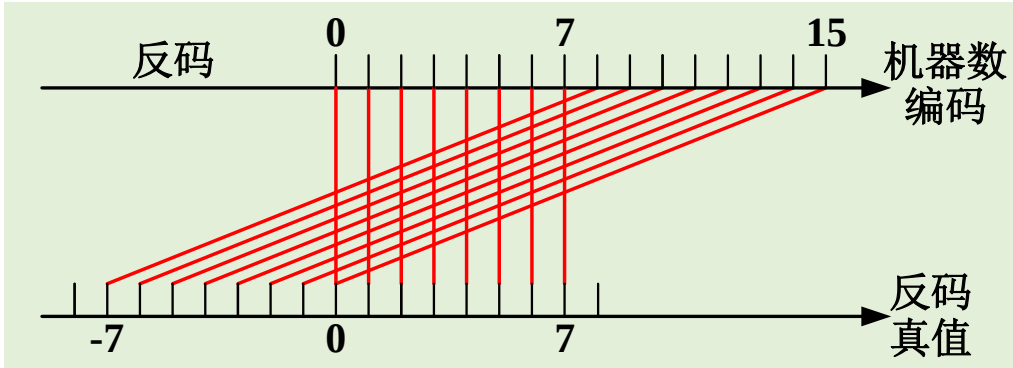
❖ 对于定点小数：

- 若 $X = +0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{反}} = 0.X_1X_2\cdots X_n$
- 若 $X = -0.X_1X_2\cdots X_n$ ，则 $[X]_{\text{反}} = 1.\overline{X_1}\overline{X_2}\cdots\overline{X_n}$



3.3.3 反码表示法

- ❖ 4位反码机器数（整数）对应的真值
- ❖ 在数轴上的表示：编码与真值的对应



机器数	原码真值	补码真值	反码真值
0000	0	0	0
0001	+1	+1	+1
0010	+2	+2	+2
0011	+3	+3	+3
0100	+4	+4	+4
0101	+5	+5	+5
0110	+6	+6	+6
0111	+7	+7	+7
1000	-0	-8	-7
1001	-1	-7	-6
1010	-2	-6	-5
1011	-3	-5	-4
1100	-4	-4	-3
1101	-5	-3	-2
1110	-6	-2	-1
1111	-7	-1	-0



❖ 例1: $X=1011$, $Y = -1011$, 则:

$$[X]_{\text{反}} = \underline{0,1011}; [Y]_{\text{反}} = \underline{1,0100};$$

❖ 例2: $X=0.1101$, $Y = -0.1101$, 则:

$$[X]_{\text{反}} = \underline{0.1101}; [Y]_{\text{反}} = \underline{1.0010};$$

❖ 例3: $X=1011$, $Y = -0.1101$, 求X和Y的8位反码机器数。

$$[X]_{\text{反}} = \underline{0,0001011}; [Y]_{\text{反}} = \underline{1.0010111};$$

❖ 例4: $[0]_{\text{反}} = ?$



- ❖ (1) 0的表示: 0 的反码表示有两种形式
 - $[+0]_{\text{反}} = 00\dots0$ $[-0]_{\text{反}} = 11\dots1$
- ❖ (2) 表示范围: 对于 $n+1$ 位反码机器数 X , 它所能表示的数据范围为:
 - 定点整数: $-(2^n - 1) \leq X \leq 2^n - 1$
 - 定点小数: $-(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$
- ❖ (3) 反码运算
 - 反码的加减运算规则: 类似于补码, 但每当有进位产生时, 必须在末位加1
 - 现代计算机已经不再使用反码机器数



❖ 1、表示方法：最高位为符号位，其他位为数值位。

- 符号位：0 - 正数，1 - 负数。
- 数值位：正数为绝对值；负数为绝对值取反。

❖ 2、0的表示：不唯一

- $[+0]_{\text{原}} = 00\dots0$ $[-0]_{\text{原}} = 11\dots1$

❖ 3、表示范围：n + 1位反码机器数X，表示范围为：

- 定点整数：- $(2^n - 1) \leq X \leq 2^n - 1$
- 定点小数：- $(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$

❖ 4、数学表示：

定点整数

$$[X]_{\text{反}} = \begin{cases} X & X \geq 0 \\ 2^{n+1} - 1 + X & X \leq 0 \end{cases}$$

定点小数

$$[X]_{\text{反}} = \begin{cases} X & X \geq 0 \\ 2 - 2^{-n} + X & X \leq 0 \end{cases}$$





四、移码表示法

❖ 1、数学定义：

定点
整数

$$[X]_{\text{移}} = 2^n + X$$

biased notation

❖ 又称增码

❖ 2、表示方法：最高位为符号位，其他位为数值位

- 符号位：1 - 正数，0 - 负数
- 数值位：正数时，与绝对值相同；负数时，为绝对值取反后，末位加1

❖ 对于定点整数：

- 若 $X = +X_1X_2\cdots X_n$ ，则 $[X]_{\text{移}} = 1X_1X_2\cdots X_n$ ；
- 若 $X = -X_1X_2\cdots X_n$ ，则 $[X]_{\text{移}} = 0\overline{X_1}\overline{X_2}\cdots\overline{X_n} + 1$ 。

移码=补码的
符号位取反



❖ 例3.6 写出下面二进制数X的移码机器数及Y的8位移码机器数。

- $X=1110$
- $[X]_{\text{移}}=1,1110$
- $X=-1110$
- $[X]_{\text{移}}=0,0001+1=0,0010$
- $Y=1110$, 真值变成7位数值
 - $Y=\underline{000}1110$
 - $[Y]_{\text{移}}=1,0001110$
- $Y=-1110$, 真值变成7位数值
 - $Y=-\underline{000}1110$
 - $[Y]_{\text{移}}=0,1110010$



四、移码表示法

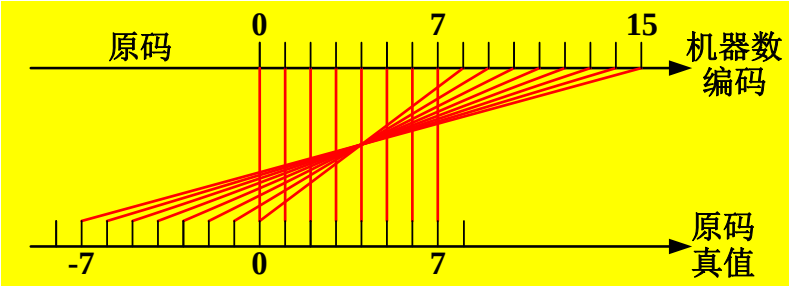
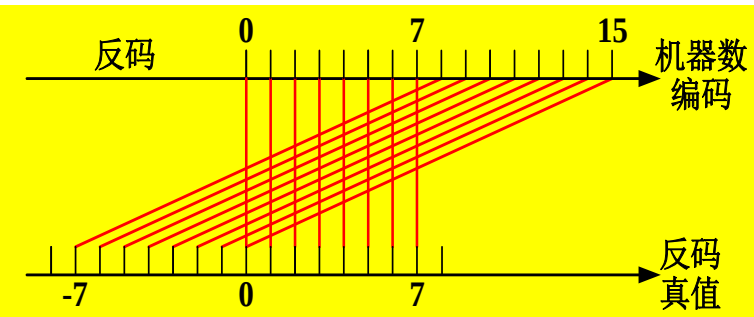
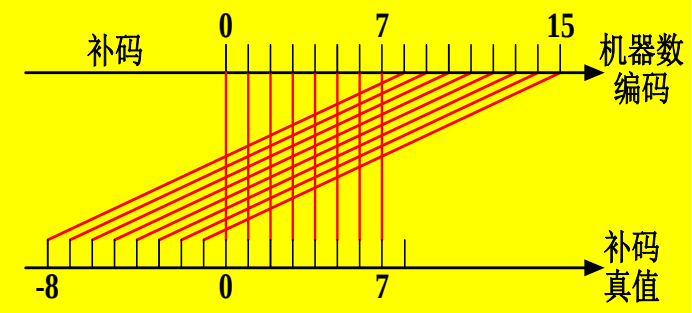
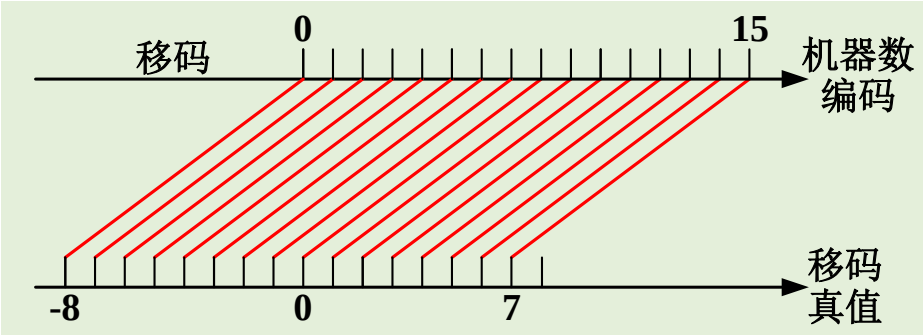
3、特点

- ❖ (1) 0的表示: 0 的移码表示形式是唯一的
 - $[+0]_{\text{移}} = 10\dots0$ $[-0]_{\text{移}} = 10\dots0$
- ❖ (2) 表示范围: 对于 $n+1$ 位移码机器数 X , 它所能表示的数据范围为:
 - 定点整数: $-2^n \leq X \leq 2^n - 1$
- ❖ (3) 移码运算:
 - 便于比较运算: 无论正负, 从符号位开始, 逐位比较大小即可
 - 加减运算也类似于补码, 但运算结果需要把符号位取反
- ❖ 移码: 通常作为浮点数的阶码



四、移码表示法

- ❖ 4位移码机器数（整数）对应的真值
- ❖ 在数轴上的表示：编码与真值的对应



机器数	原码真值	补码真值	反码真值	移码真值
0000	0	0	0	-8
0001	+1	+1	+1	-7
0010	+2	+2	+2	-6
0011	+3	+3	+3	-5
0100	+4	+4	+4	-4
0101	+5	+5	+5	-3
0110	+6	+6	+6	-2
0111	+7	+7	+7	-1
1000	-0	-8	-7	0
1001	-1	-7	-6	+1
1010	-2	-6	-5	+2
1011	-3	-5	-4	+3
1100	-4	-4	-3	+4
1101	-5	-3	-2	+5
1110	-6	-2	-1	+6
1111	-7	-1	-0	+7



四种定点机器数的对比

1、表示方法

定点整数

真值	$X = + X_1 X_2 \dots X_n$	$X = - X_1 X_2 \dots X_n$
原码	$[X]_{\text{原}} = 0 X_1 X_2 \dots X_n$	$[X]_{\text{原}} = 1 X_1 X_2 \dots X_n$
反码	$[X]_{\text{反}} = 0 X_1 X_2 \dots X_n$	$[X]_{\text{反}} = 1 \overline{X_1} \overline{X_2} \dots \overline{X_n}$
补码	$[X]_{\text{补}} = 0 X_1 X_2 \dots X_n$	$[X]_{\text{补}} = 1 \overline{X_1} \overline{X_2} \dots \overline{X_n} + 1$
移码	$[X]_{\text{移}} = 1 X_1 X_2 \dots X_n$	$[X]_{\text{移}} = 0 \overline{X_1} \overline{X_2} \dots \overline{X_n} + 1$

定点
小数

真值	$X = +0.X_1 X_2 \dots X_n$	$X = -0.X_1 X_2 \dots X_n$
原码	$[X]_{\text{原}} = 0.X_1 X_2 \dots X_n$	$[X]_{\text{原}} = 1.X_1 X_2 \dots X_n$
反码	$[X]_{\text{反}} = 0.X_1 X_2 \dots X_n$	$[X]_{\text{反}} = 1.X_1 X_2 \dots X_n$
补码	$[X]_{\text{补}} = 0.X_1 X_2 \dots X_n$	$[X]_{\text{补}} = 1.\overline{X_1} \overline{X_2} \dots \overline{X_n} + 0.00 \dots 1$
移码	$[X]_{\text{移}} = 1.X_1 X_2 \dots X_n$	$[X]_{\text{移}} = 0.\overline{X_1} \overline{X_2} \dots \overline{X_n} + 0.00 \dots 1$



真值	$X = +0$	$X = -0$	
原码	$[X]_{\text{原}} = 0\ 00\dots0$	$[X]_{\text{原}} = 1\ 00\dots0$	不唯一
反码	$[X]_{\text{反}} = 0\ 00\dots0$	$[X]_{\text{反}} = 1\ 11\dots1$	不唯一
补码	$[X]_{\text{补}} = 0\ 00\dots0$		唯一
移码	$[X]_{\text{移}} = 1\ 00\dots0$		唯一



$n+1$ 位机器数

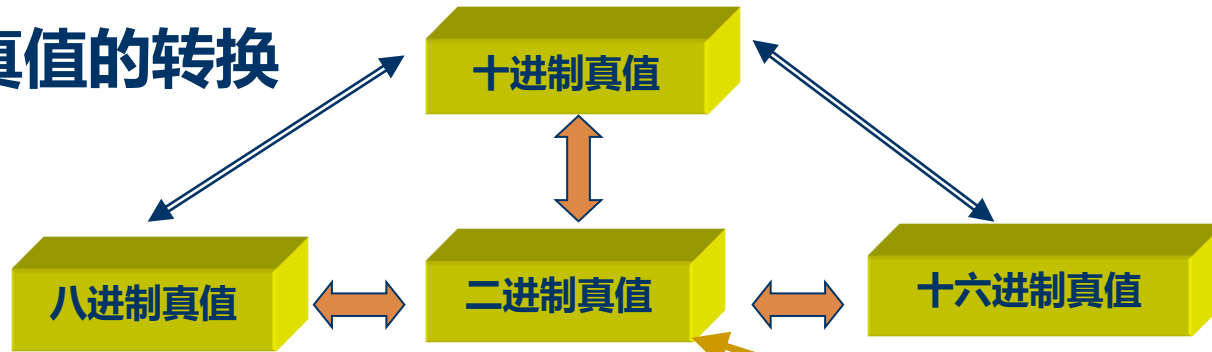
	定点整数	定点小数
原码	$-(2^n - 1) \leq X \leq 2^n - 1$	$-(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$
反码	$-(2^n - 1) \leq X \leq 2^n - 1$	$-(1 - 2^{-n}) \leq X \leq 1 - 2^{-n}$
补码	$-2^n \leq X \leq 2^n - 1$	$-1 \leq X \leq 1 - 2^{-n}$
移码	$-2^n \leq X \leq 2^n - 1$	$-1 \leq X \leq 1 - 2^{-n}$



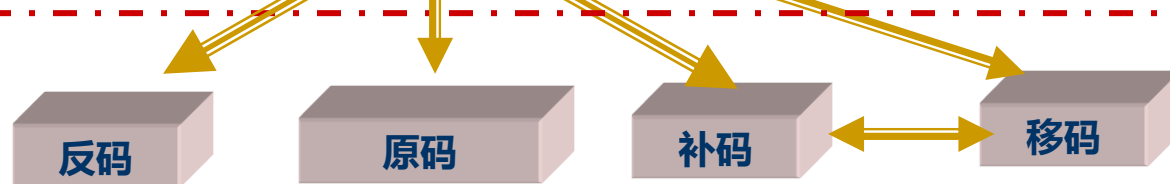


机器数→真值

不同进制真值的转换



机器码的转换关系



❖ 机器数转换为真值

- ①机器数的符号位→真值的正负
- ②机器数的定义和编码方法→真值的绝对值

❖ 机器数之间的相互转换

- 最简单的方法：先求出它们的真值，然后再转换为另一种表示方法。



定点整数：机器数→真值

❖ $[X]_{\text{原}} \rightarrow \text{真值?}$

❖ $[X]_{\text{原}} = X_S, X_1 X_2 \dots X_n$

- $X_S = 0: X = +X_1 X_2 \dots X_n$
- $X_S = 1: X = -X_1 X_2 \dots X_n$

❖ $[X]_{\text{补}} \rightarrow \text{真值?}$

❖ $[X]_{\text{补}} = X_S, X_1 X_2 \dots X_n$

- $X_S = 0: X = +X_1 X_2 \dots X_n$
- $X_S = 1: X = -(\overline{X_1} \overline{X_2} \dots \overline{X_n} + 1)$

❖ $[X]_{\text{反}} \rightarrow \text{真值?}$

❖ $[X]_{\text{反}} = X_S, X_1 X_2 \dots X_n$

- $X_S = 0: X = +X_1 X_2 \dots X_n$
- $X_S = 1: X = -\overline{X_1} \overline{X_2} \dots \overline{X_n}$

❖ $[X]_{\text{移}} \rightarrow \text{真值?}$

❖ $[X]_{\text{移}} = X_S, X_1 X_2 \dots X_n$

- $X_S = 0: X = -(\overline{X_1} \overline{X_2} \dots \overline{X_n} + 1)$
- $X_S = 1: X = +X_1 X_2 \dots X_n$



例题：

❖ 例3.7 已知X的机器数如下，请写出X的二进制真值。

❖ $[X]_{\text{原}} = 1,0000000$

❖ $[X]_{\text{补}} = 1,0000000$

❖ $[X]_{\text{移}} = 1,0000000$

❖ $[X]_{\text{反}} = 1,0000000$

❖ $[X]_{\text{原}} = 0,1101$

❖ $[X]_{\text{补}} = 0,1101$

❖ $[X]_{\text{反}} = 0,1101$

❖ $[X]_{\text{移}} = 0,1101$

❖ $[X]_{\text{原}} = 1,1000$

❖ $[X]_{\text{补}} = 1,1000$

❖ $[X]_{\text{反}} = 1,1000$

❖ $[X]_{\text{移}} = 1,1000$



❖ $X = -0$

❖ $X = - (1111111 + 1)_2 = (-128)_{10}$

❖ $X = 0$

❖ $X = (-1111111)_2 = (-127)_{10}$

❖ $X = +1101$

❖ $X = +1101$

❖ $X = +1101$

❖ $X = - (0010 + 1) = -0011$

❖ $X = -1000$

❖ $X = - (0111 + 1) = -1000$

❖ $X = -0111$

❖ $X = +1000$





3.4 浮点机器数表示方法

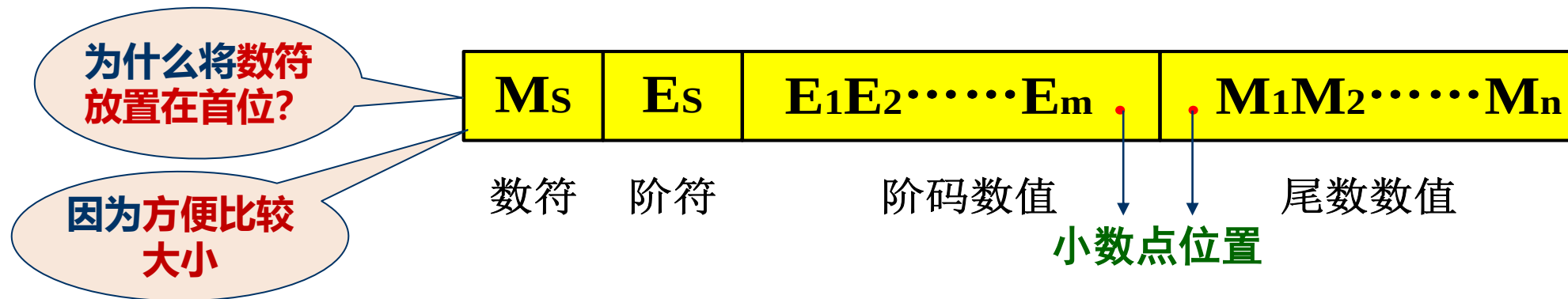
- ❖ 3.4.1 浮点数的格式
- ❖ 3.4.2 浮点数的规格化表示
- ❖ 3.4.3 浮点数的表示范围
- ❖ 3.4.4 IEEE 754浮点数标准





3.4.1 浮点数的格式

- ❖ 浮点数N的构成: $N = M \times R^E$
- ❖ 阶码E (Exponent) 、尾数M (Mantissa) 和阶码的底R (Radix)
- ❖ 阶码的底R: 是一个常数, 隐含规定, 一般为2、8或16



- ❖ 浮点数格式要素:
 - ①位数: 阶码位数+尾数位数→总位数
 - ②编码: 阶码和尾数采用的机器数编码
 - ③排列顺序: 数符是否在最高位
 - ④其他特殊的编码规则: 譬如隐藏位



3.4.1 浮点数的格式

❖ 尾数M：定点小数

$$0.101110 \times 2^{-2} = 0.0010111$$

- 尾数的位数：决定了浮点数有效数值的精度
- 尾数的符号：数符，代表了浮点数的正负
- 尾数的编码：一般采用原码和补码表示

$$N = M \times R^E$$

❖ 阶码E：定点整数

- 阶码的位数多少：决定了浮点数的表示范围。
- 阶码的数值大小：决定了数据实际小数点位置与尾数的小数点位置之间的偏移量
- 阶码的符号：阶符，决定了实际小数点位置在尾数的小数点位置的左边还是右边
- 阶码的编码：一般采用移码和补码表示

❖ 阶码的底R：一般为2、8或16，且隐含规定





3.4.2 浮点数的规格化表示

1、浮点数的规格化表示：为了充分利用，将尾数的绝对值限定在某个范围之内

- 目的：充分利用尾数的二进制数位来表示更多的有效数字

- 例如： $0.0010111 = 0.010111 \times 2^{-1} = 0.\underline{1}01110 \times 2^{-2}$

❖ $R = 2$ ，则规格化浮点数的尾数 M 应满足条件：绝对值最高有效位为1，即

$$\frac{1}{2} \leq |M| \leq 1$$

没有前导零

2、计算机硬件如何判断：浮点数是否规格化？

- 原码表示的尾数：

- $M_1 = 1$ ：规格化，即尾数为 $\times.1 \times \dots \times$ 形式

- 补码表示的尾数：

- $M_s \oplus M_1 = 1$ ：规格化，即尾数为 $0.1 \times \dots \times$ 或 $1.0 \times \dots \times$ 形式

❖ 尾数=-0.5？

❖ 用原码表示是规格化的

❖ 用补码表示是非规格化



3.4.2 浮点数的规格化表示

❖ 非规格化浮点数:

- 规格化操作: 修改阶码和左右移尾数的方法来使其变为规格化浮点数
- 右规: 尾数进行右移实现的规格化
- 左规: 尾数进行左移实现的规格化

阶码?

阶码?

❖ (自定义格式) 规格化浮点数表示方法:

- (1) 写出数据的二进制真值
- (2) 转换为 $M \times 2^E$ 的形式, 其中 M 为没有前导零的定点小数, E 为整数
- (3) 按照格式写出 M 和 E 的规定机器数编码
- (4) 按照格式要求排列 E 和 M

❖ 什么情况下要做规格化操作?

- 尾数有前导零: 即 $< 1/2$, 需要左规
- 尾数运算溢出: 即 ≥ 1 , 需要右规

❖ 使用规格化浮点数表示数据的优点:

- 提高了浮点数据的精度
- 使程序能够更方便地交换浮点数据
- 可以使浮点数的运算更为简化



3.4.2 浮点数的规格化表示

❖ 例3.8 某机浮点数表示格式为：16位浮点数，阶码6位，含1位阶符，用移码表示；尾数10位，包含1位数符，用补码表示；阶码在前，尾数(包括数符)在后

- (1) 请写出 $X = -27/128$ 的规格化浮点数形式。
- (2) Y是这种格式的浮点数，其编码用16进制表示为99BEH，请求出其真值。

阶码(6位)	尾数(10位), 含数符1位
--------	----------------

❖ (1) X真值→规格化浮点数

① 写出数据的二进制真值： $X = -27/128 = (-0.0011011)_2$

② 转换为 $M \times 2^E$ (M没有前导零)： $X = -0.11011 \times 2^{-10}$

③ 写出M和E的规定机器数编码： $E_X = -00010$ $M_X = -0.110110000$

$[E_X]_{\text{移}} = 0,11110$

$[M_X]_{\text{补}} = 1.00101000$

④ 按格式排列： $[X]_{\text{浮}} = 0,11110 \ 1.001010000B = 7A50H$



3.4.2 浮点数的规格化表示

❖ 例3.8

- (2) Y是这种格式的浮点数，其编码用16进制表示为99BEH，请求出其真值。

阶码(6位)

尾数(10位)，含数符1位

❖ (2) Y的浮点数编码→真值

①写出浮点数的二进制编码： $[Y]_{\text{浮}} = 99BEH = 1001\ 1001\ 1011\ 1110B$

②按照浮点数格式析出阶码E和尾数M的机器数：

$$[E_Y]_{\text{移}} = 1,00110$$

$$[M_Y]_{\text{补}} = 0.110111110$$

③求出阶码E和尾数M的真值：

$$E_Y = +00110$$

$$M_Y = +0.110111110$$

④写出对应真值 $= M \times 2^E$

$$Y = +0.110111110 \times 2^{+00110} = +110111.11B = +55.75$$





3.4.3 浮点数的表示范围

❖ 表示范围：[最小数，最大负数]，0，[最小正数，最大数]

浮点数	尾数M	阶码E
最小数	最小数	最大数 ?
最大负数	最大负数	最小数 ?
最小正数	最小正数	最小数
最大数	最大数	最大数

$$N = M \times R^E$$

❖ 四个数大小与什么有关？

- 阶码的底R
- 阶码和尾数的位数
- 采用何种机器数编码

❖ 浮点数的真值零：表示数轴上的0点

❖ 浮点数的机器零：指数轴上0点及其附近的一段区域

- 两种情况计算机将其当做零处理（使用真值零的编码）
- ①浮点数的尾数为零
- ②阶码负溢出



3.4.3 浮点数的表示范围

❖ 例3.9 写出例3.8浮点数格式的规格化和非规格化表示范围

▪ 阶码6位、移码表示；尾数为10位、补码表示

❖ 规格化表示范围：
尾数必须规格化

	真值	浮点数表示
最小数	$-1 \times 2^{31} = -2^{31}$	1, 11111 1.000000000
最大负数	$-(2^{-1} + 2^{-9}) \times 2^{-32}$	0, 00000 1.011111111
最小正数	$2^{-1} \times 2^{-32} = 2^{-33}$	0, 00000 0.100000000
最大数	$(1 - 2^{-9}) \times 2^{31}$	1, 11111 0.111111111

❖ 非规格化表示范围：

	真值	浮点数表示
最小数	$-1 \times 2^{31} = -2^{31}$	1, 11111 1.000000000
最大负数	$-2^{-9} \times 2^{-32} = -2^{-41}$	0, 00000 1.111111111
最小正数	$2^{-9} \times 2^{-32} = 2^{-41}$	0, 00000 0.000000001
最大数	$(1 - 2^{-9}) \times 2^{31}$	1, 11111 0.111111111



- ❖ 等长的浮点机器数和定点机器数：
 - 浮点数的表示范围一定比定点数大，而有效数值的精度却低于定点数
- ❖ 等长的浮点机器数：
 - 阶码位数越长，表示范围也越广泛，相对精度受到影响
 - 尾数位数越长，表示的精度越高，但表示范围相比会缩小
 - 必须协调好浮点数的阶码和尾数位数，以保证在有限位数的前提下，既扩大数的表示范围，又能保持适当的表示精度
- ❖ “大数吃小数” 的现象：由浮点数的疏密不均导致
- ❖ 有可能： $(\text{大数}a + \text{小数}b) - \text{大数}a \neq (\text{大数}a - \text{大数}a) + \text{小数}b$
- ❖ 左侧=0（小数b可能被大数a所吸收），右侧=小数b



- ❖ 实践题：请运行下面程序，打印输出进行观察
 - 分析大数多大、小数多小才会产生这种现象？通过分析及编程实践，总结一般性规律和条件

```
#include <stdlib.h>
int main() {
    float f1=10000000;
    float f2=0.5;
    float f3=(f1+f2)-f1;
    float f4=(f1-f1)+f2;
    printf("f3 = %f\n",f3);
    printf("f4 = %f",f4);
    return(0);
}
```

➡ **Output** Empty

标准输出：

```
f3 = 0.000000
f4 = 0.500000
```





3.4.4 IEEE 754浮点数标准

❖ 1、IEEE 754 浮点数格式

- IEEE 754 : 浮点数国际标准, $R=2$, 格式有4种

排列、位数和编码

短实数

长实数

类型	总位数	数符 M_s	阶码E $E_s E_1 \dots E_m$	尾数数值 $M_1 \dots M_n$
单精度(float)	32	1位	8位	23位
双精度(double)	64	1位	11位	52位
扩展单精度	≥ 43	1位	≥ 11	≥ 31 位
扩展双精度	≥ 79	1位	≥ 15	≥ 63 位
编码		原码	2^n-1 的移码	原码

❖ Intel x86 FPU: 实现80位的扩展双精度浮点数

❖ 阶码: 15位, $2^{14}-1=16383$ 的移码

❖ 尾数: 64位, 1位整数+63位小数, 原码表示



3.4.4 IEEE 754浮点数标准

- ❖ 2、IEEE754浮点数约定与特殊值
- ❖ 数据分为三类：规格化数、非规格化数和特殊数值
 - 以单精度浮点数为例：

Ms	E	M	意义
0/1	0	0	± 0
0/1	0	非0	非规格化数
0/1	1~254	任意	规格化数
0/1	255	0	$\pm$ 无穷大
0/1	255	非0	NaN



3.4.4 IEEE 754浮点数标准

❖ 2、IEEE754浮点数约定与特殊值

❖ 数据分为三类：规格化数、非规格化数和特殊数值

❖ 单精度

Ms	E	M	数据
0	00000000	00.....00	+0
0	00000000	00.....01	最小非规格化正数
0			非规格化正数
0	00000000	11.....11	最大非规格化正数
0	00000001	00.....00	最小规格化正数
0			规格化正数
0	11111110	11.....11	最大规格化正数
0	11111111	00.....00	$+\infty$
0	11111111	00.....01	NaN
0	11111111		NaN
0	11111111	11.....11	NaN

Ms	E	M	数据
1	00000000	00.....00	-0
1	00000000	00.....01	最大非规格化负数
1			非规格化负数
1	00000000	11.....11	最小非规格化负数
1	00000001	00.....00	最大规格化负数
1			规格化负数
1	11111110	11.....11	最小规格化负数
1	11111111	00.....00	$-\infty$
1	11111111	00.....01	NaN
1	11111111		NaN
1	11111111	11.....11	NaN



3.4.4 IEEE 754浮点数标准 2、IEEE754浮点数约定与特殊值

❖ (1) 规格化数：阶码E既非全0，也非全1

表示范围：
 $-(2^n-1) \sim 2^n$

单精度：
 $-127 \sim 128$

- 阶码：是 2^n-1 的移码， $E=[E']_{移}=2^n-1+E'$
- 隐藏位：单、双精度浮点数，尾数规格化的“1”，放置于整数位，并将其隐藏；临时实数无隐藏位
- 符号位：尾数的符号位Ms在最高位

隐藏位,位于整数位

类型	真值计算
单精度	规格化: $N = (-1)^{M_s} \times (1.M_1M_2...M_n) \times 2^{E-127}$ 非规格化: $N = (-1)^{M_s} \times (0.M_1M_2...M_n) \times 2^{-126}$
双精度	规格化: $N = (-1)^{M_s} \times (1.M_1M_2...M_n) \times 2^{E-1023}$ 非规格化: $N = (-1)^{M_s} \times (0.M_1M_2...M_n) \times 2^{-1022}$



3.4.4 IEEE 754浮点数标准 2、IEEE754浮点数约定与特殊值

❖ (2) 非规格化数：阶码 $E=0$

- 阶码真值=最小值=1-偏移量 尾数没有隐藏位
- 作用1：表示 ± 0.0 ， $M=全0$
- 作用2：用于表示接近真值0的、绝对值很小的数据（在0和距离最近的规格化数之间）
- 非规格化数、规格化数的真值计算方法不同
- 非规格化数 平滑过渡到 规格化数：两者边界上，编码及真值都是相邻的
- 最大非规格化正数和最小规格化正数的编码相邻，对应真值分别是 $(1-2^{-23}) \times 2^{-126}$ 和 1.0×2^{-126} ，它们在数轴上也是相邻的点



3.4.4 IEEE 754浮点数标准 2、IEEE754浮点数约定与特殊值

❖ (3) 特殊数值：阶码E为全1（最大值）

■ ①无穷大：尾数M为全0

- $M_s=0$ ，表示正无穷大； $M_s=1$ ，表示负无穷大

■ ②NaN：尾数M不是全0

- 非数（Not a Number），不是一个数：未定义或不可表示的值
- 例如：对0和无穷大进行的算术运算、对负数开偶次方等操作

真值	M_s	E	M	计算值
+0	0	00000000	00.....00	0.0×2^{-127}
-0	1	00000000	00.....00	-0.0×2^{-127}
$+\infty$	0	11111111	00.....00	$1.0 \times 2^{+128}$
$-\infty$	1	11111111	00.....00	$-1.0 \times 2^{+128}$
NaN	X	11111111	≠全零	不是一个数



3.4.4 IEEE 754浮点数标准

❖ 真值→求得单精度、双精度浮点数:

- ① 写出数据的二进制真值
- ② 转换为 $M' \times 2^{E'}$ 的形式, 其中 $M' = \pm 1.\times \times \dots \times$ 形式的定点小数, E' 为整数
- ③ 写出尾数 M' 的原码, 且隐藏 (舍去) 整数位的 “1” :

正数 $M_s = 0$, 负数 $M_s = 1$

$M = M_1 M_2 \dots M_n = \times \times \dots \times$

- ④ 写出阶码 E' 的移码:
 - 单精度: $[E']_{\text{移}} = E' + 127 = E$
 - 双精度: $[E']_{\text{移}} = E' + 1023 = E$
- ⑤ 按照格式排列 E 和 M
 - $M_s \quad E \quad M$



3.4.4 IEEE 754浮点数标准

- ❖ 例3.10 若X和Y均是IEEE 754标准的单精度浮点数。
 - (1) 若 $X = -135.625$ ，求X的规格化浮点数表示。
 - (2) 若浮点数Y的存储形式为41360000H，求Y的真值。
- ❖ (1) 真值→IEEE 754规格化单精度浮点数
- ❖ ①写出数据的二进制真值 $X = (-10000111.101)_2$
- ❖ ②转换为 $M' \times 2^{E'}$ 的形式: $M' = \pm 1.\times \times \dots \times$, E' 为整数
 - $X = -1.0000111101 \times 2^7$ $M' = -1.0000111101$ $E' = +7$
- ❖ ③写出尾数 M' 的原码，尾数数值隐藏（舍去）整数位的“1”
 - $M_s = 1$ 尾数数值 $M = 000\ 0111\ 1010\ 0000\ 0000\ 0000$
- ❖ ④写出阶码 E' 的移码 E : $E = [E']_{\text{移}} = 127 + E' = 127 + 7 = 134 = 1000\ 0110\text{B}$
- ❖ ⑤按照格式排列E和M: 即 $M_s\ E\ M$
 - $[X]_{\text{浮}} = 1\ 10000110\ 000\ 0111\ 1010\ 0000\ 0000\ 0000\text{B} = \text{C307A000H}$



3.4.4 IEEE 754浮点数标准

❖ 例3.10 (2) 浮点数→真值

阶码8位, $2^7-1=127$ 的移码

- ① 浮点数的二进制编码: 按照格式拆分, 写出 M_S 、阶码E和尾数数值M

• $[Y]_{\text{浮}} = 41360000H = \text{0100 0001 0011 0110 0000 0000 0000 0000 B}$

• $M_S = \text{0}$ $E = \text{1000 0010 B} = 130$ $M = \text{011 0110 0000 0000}$

• 阶码编码E不是全0也不是全1, 可判定Y是规格化浮点数

添加隐藏位1

② 写出尾数的真值 M' : $M_S = \text{0}$, 表明是正数 添加隐藏位1

• $M' = +1.011 0110 0000 0000 0000 0000 B$

尾数数值位
23位原码

数符+

③ 求出阶码的真值 E' : Y是单精度浮点数, 故 $E' = E - 127 = 130 - 127 = 3$

- ④ 按照 $M' \times 2^{E'}$, 求出浮点数的真值

• $Y = M' \times 2^{E'} = +1.011 0110 0000 0000 0000 0000 \times 2^3$
 $= +1011.011 B = +11.375$



3.4.4 IEEE 754浮点数标准

- ❖ 练习：C语言定义并赋值了变量：float x=0.1;
- ❖ x在内存中的二进制代码是：0011 1101 1100 1100 1100 1100 1100 1101
- ❖ 请计算出其对应的二进制真值，判断它是>0.1还是<0.1?

❖ Ms=0, 表明正数

❖ $[E']_{\text{移}} = 011\ 1101\ 1B = 123 = 127 + E'$

■ 则：阶码的真值 $E' = 123 - 127 = -4$

❖ $M' = +1.100\ 1100\ 1100\ 1100\ 1100\ 1101\ b$

❖ $x = +1.100\ 1100\ 1100\ 1100\ 1100\ 1101 \times 2^{-4}$
 $= +0.0001100\ 1100\ 1100\ 1100\ 1100\ 1101b$

$x = 13421773 / 2^{27} = 13421773 / 134,217,728$
 $= 0.100000001490116119384765625$





浮点数表示的误差

- ❖ 1991年2月25日（海湾战争），美军位于沙特的一座军营，被伊拉克军队发射的一枚“飞毛腿”导弹击中。而美军爱国者导弹未能成功拦截，造成美军28人当场死亡，100多人受伤，是美军在海湾战争中最大的一次伤亡事件。
- ❖ 原因是：数据0.1的表示误差所致，酿成了灾难性的后果。

为何爱国者导弹愣没把飞毛腿导弹拦住？





浮点数表示的误差

❖ 美国总审计局的分析报告指出：失败原因是一个数字计算不精确

❖ 爱国者导弹系统

用一个计数器来记录时间
每0.1秒计数器+1 (初始为0)



秒数=计数器值 n *0.1

❖ 0.1如何

浮点数表示有精度误差，不能准确表示每一个实数

经过多次运算后，也可能产生累积误差

作为计算机专业工程师，特别要重视细节

❖ 雷达发现

0

❖ 程序计算得到的时间误差 = $n * \delta \approx 0.34344s$

❖ 飞毛腿导弹速度是1676米/秒，则系统预测目标出现位置的误差=?

❖ $1676 * 0.34344 \approx 576$ 米

失之毫厘，谬以千里



浮点数表示的误差

- ❖ **探索题：**伊利诺伊大学厄巴纳-香槟分校的计算机科学教授Robert Skeel对美国总审计局的分析报告提出质疑：预测来袭导弹下一时刻出现的位置，应该使用两个时间点的计数器差值乘以导弹飞行速度，得到导弹的飞行距离，进而计算预期位置。
- ❖ 计算误差应该依赖于两次连续读取之间的相对时间，而不是绝对时间，这样，假设两个时间点相隔1秒，误差只有3-4毫米，完全不影响系统的正常拦截。
- ❖ 有工程师发现0.1误差存在的潜在问题，软件也因此升级过，将部分模块的0.1用48位的小数表示。他提出由于软件更新不完全，提供这两个时间点的程序，使用不同位数的0.1，从而引起时间误差。



浮点数表示的误差

- ❖ 按照这个思路进行假设，完成下面的计算，分析是否可导致拦截失败。
- ❖ (1) 系统已运行100小时，当前时刻是 t_1 ，使用24位的0.1来计算，当前时间是多少秒？
- ❖ (2) 下一时刻 t_2 是1秒钟后，使用48位的0.1来计算（误差可以当做0）， t_2 时刻的时间是多少秒？两个时间点差值 $t_2 - t_1$ 和真实的1秒时间的误差是多少？
- ❖ (3) 用两个时间点差值 $t_2 - t_1$ ，乘以导弹飞行速度1676米/秒，可得到预测的导弹飞行距离，计算预测距离的误差是多少米？
- ❖ **实践题：**针对前述两种假设，如果数据0.1采用IEEE 754标准的单精度浮点数表示，请用高级语言编写程序，计算相关的数据，再次分析会不会导致拦截失败。





3.5 非数值数据的表示

- ❖ 非数值数据：文字和符号（**字符**）、图像、声音等
- ❖ 非数值数据的表示：对其进行二进制编码

❖ 3.5.1 字符编码

❖ 3.5.2 汉字编码





3.5.1 字符编码

- ❖ **字符的表示：**采用字符编码，用规定的二进制数表示文字和符号
- ❖ **ASCII码：**美国信息交换标准代码，由7位二进制表示，可表示128种字符：
 - **95个可打印或显示的字符：**
 - 10个数字 '0' ~ '9': 30H ~ 39H, 顺序排列■
 - 26个小写字母 'a' ~ 'z': 61H ~ 7AH, 顺序排列■
 - 26个大写字母 'A' ~ 'Z': 41H ~ 5AH, 顺序排列■
 - 各种运算符号和标点符号等
 - **33个控制字符：不可打印或显示**
 - ① 10个传输类控制字符：用于数据传输控制； ■
 - ② 6个格式类控制字符，用于控制数据的位置 ■
 - ③ 4个设备类控制字符，用于控制辅助设备； ■
 - ④ 4个信息分隔类控制字符，用于分隔或限定数据■
 - ⑤ 9个其他控制字符、空格字符和删除字符



ASCII码编码表

编码	B ₃ B ₂ B ₁ B ₀															
B ₆ B ₅ B ₄	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
000	NUL	SOH	STX	ETX	EOT	ENQ	ACK	DEL	BS	HT	LF	VT	FF	CR	SO	SI
001	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
010	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
011	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
100	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
101	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	↑	_
110	,	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
111	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL



3.5.2 汉字编码

❖ 计算机对汉字的处理技术，必须解决三个问题：

- 汉字输入 → ■ 汉字输入码
- 汉字储存与交换 → ■ 内码、交换码
- 汉字输出 → ■ 字形码

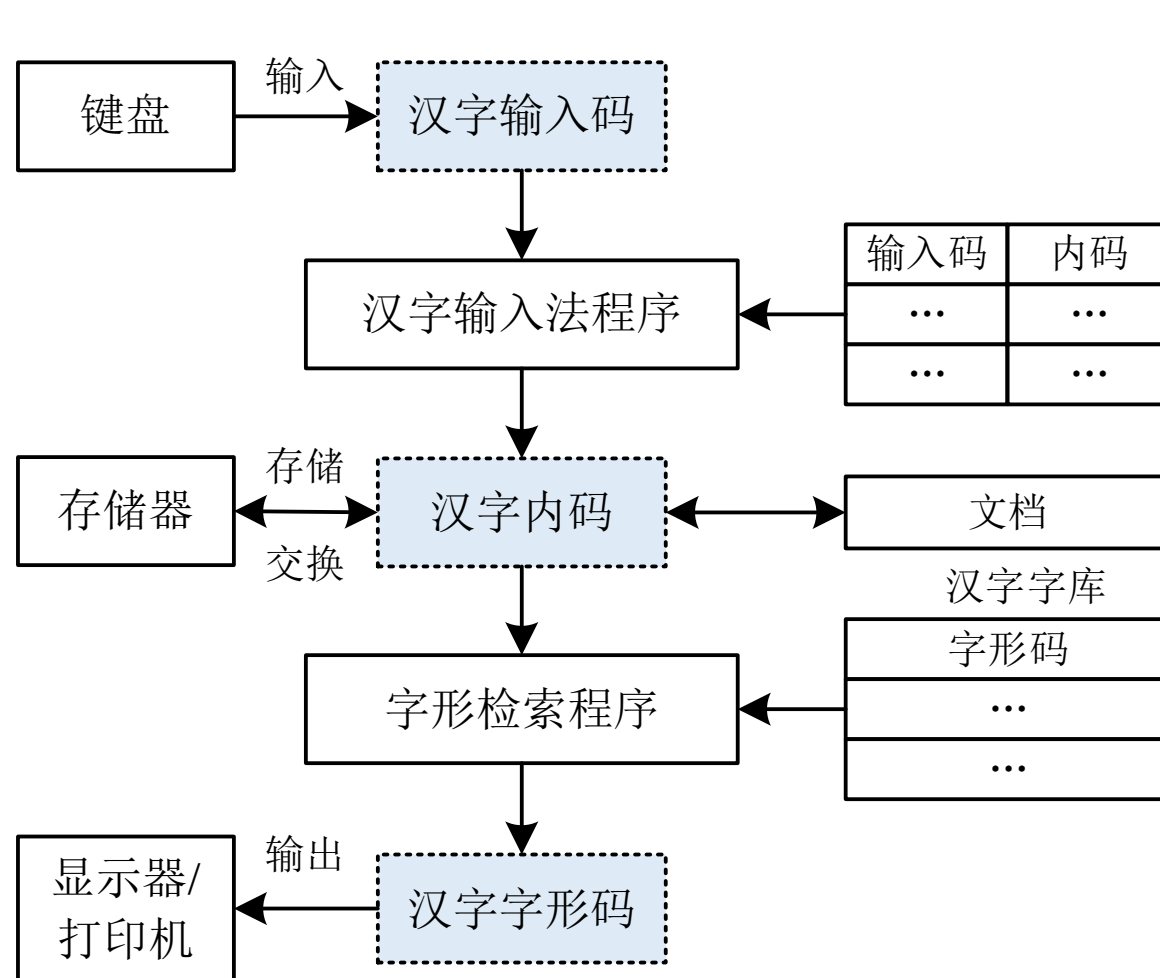
❖ 1、汉字输入码

❖ 2、汉字内码

❖ 3、字形码

❖ 4、汉字标准

❖ 5、汉字印刷





1、汉字输入码

- ❖ **汉字输入：**在西文标准键盘上，通过不同的字符组合，将汉字输入计算机，转换为汉字内码的过程
- ❖ **汉字输入码：**也称**外码**，由不同的西文字符组合成的编码
- ❖ **汉字输入码种类：**
 - **数字编码：**区位码、国标码、电报码等
 - **拼音编码：**搜狗拼音、微软拼音等
 - **字形编码：**五笔字型输入法、郑码
 - **音形编码：**自然码、钱码





2、汉字内码

- ❖ **汉字内码：用于汉字信息的存储、交换、检索等操作的机内代码**
 - **一般采用两个字节表示，汉字内码在计算机中是唯一的**
- ❖ **汉字内码涉及以下几种编码：**
- ❖ **(1) 区位码**
 - **汉字编码的国家标准：GB2312-1980《信息交换用汉字编码字符集·基本集》**
 - **共收录了6763个汉字、682个图形符号**
 - **一级常用汉字：3755个，按拼音字母顺序排列**
 - **二级次常用汉字：3008个，按偏旁部首排列**
 - **两个字节编码汉字，每个字节各取7位，可编码 $128 \times 128 = 16384$ 个字符**
 - **将所有的汉字及符号组成一个 94×94 的方阵，行→“区”，列→“位”**
 - **每个汉字与符号都位于某个区的某个位上，区号和位号→“区位码”**
 - **例如：“中”字位于54区48位→“中”字的区位码即为“5448”**



2、汉字内码

❖ (2) 国标码

- 国标码 = 区位码 + 2020H, 占用两个字节
- 例如 “中” 字: 区位码 = 5448
 - 区码和位码转化为16进制, = 3630H
 - 国标码 = 3630H + 2020H = 5650H

❖ (3) 汉字内码

- 西文字符: 七位的ASCII码, 当用一个字节表示时, 最高位为 “0”
- 汉字内码: 汉字内码 = 汉字国标码 + 8080H, 2个字节的最高位均为 “1”
 - 例如 “中” 字的机内码 = 5650H + 8080H = D6D0H
- 文本文件中储存的是汉字内码





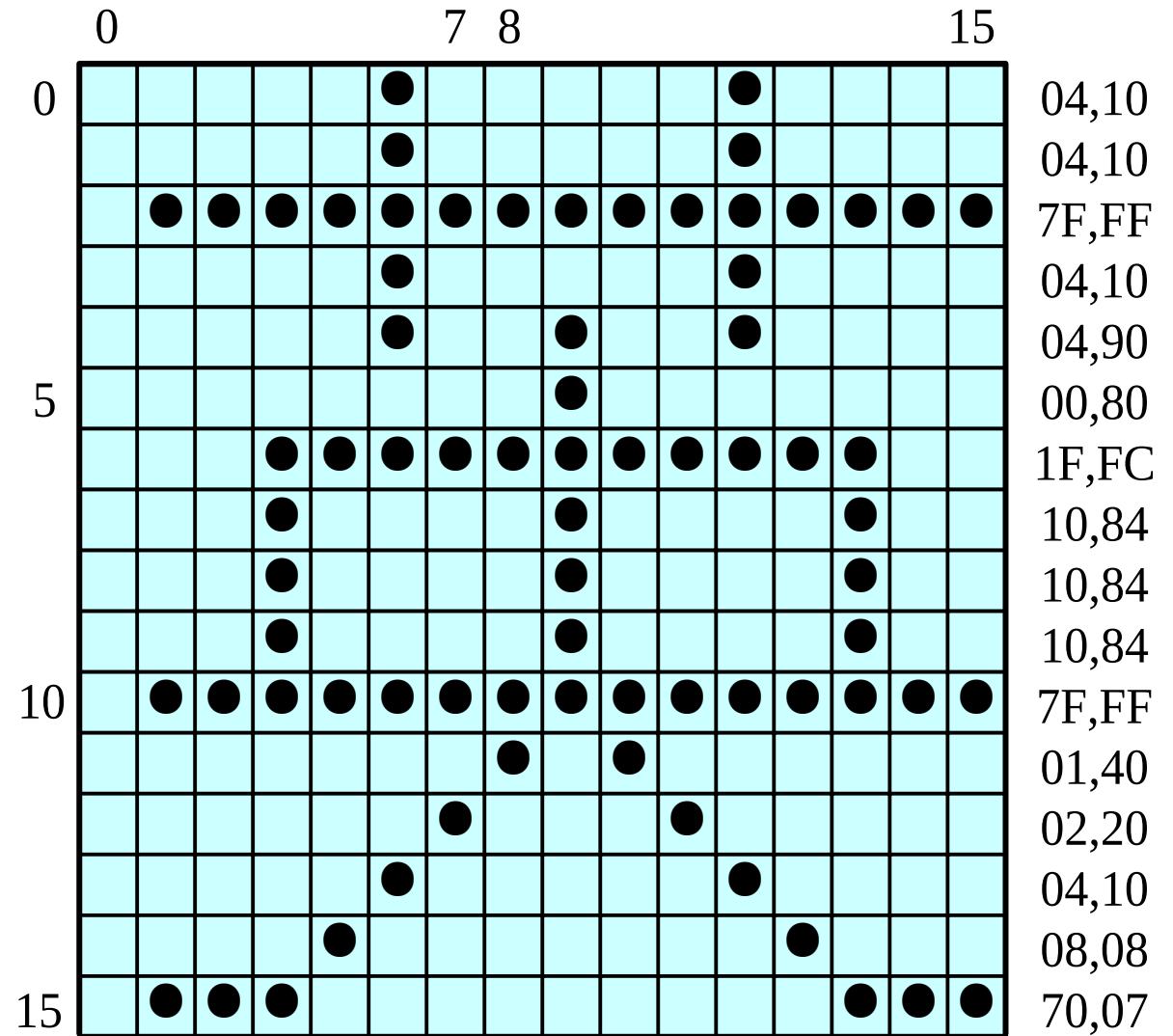
3、汉字字形码

- ❖ **汉字处理系统中字形信息：**①用活字或文字版的母体字形形式；②点阵表示法、矢量表示法等形式。最基本的，应用也最广泛的是**字模码**
 - **字模码：**用点阵形式表示与存储汉字字形代码，它是汉字的输出形式，用于汉字的显示和打印
 - 根据汉字输出的要求不同，点阵的规模也不同：
 - 简易型汉字：16×16， 32字节/汉字
 - 普及型汉字：24×24， 72字节/汉字
 - 提高型汉字：32×32， 128字节/汉字
 - 精密型汉字：48×48， 288字节/汉字
 - **汉字字库：**将所有汉字的字模点阵代码按顺序集中存放，构成了汉字库
 - **字形检索程序：**将汉字的机内代码转化为汉字的字形码



3、汉字字形码

- ❖ **16×16点阵**: $16 \times 16b = 32B/\text{汉字}$
- ❖ **汉字点阵字形的汉字库**: 结构简单, 但是当需要对汉字进行放大、缩小、平移、倾斜、旋转、投影等变换时, 汉字的字形效果就欠佳
- ❖ 可使用**矢量汉字库**、**曲线字库**的汉字, 其字形用直线或曲线表示, 能产生高质量的输出字形





汉字编码之间的关系





4、汉字标准

❖ 国家标准：

- **GB2312-1980**：收录了6763个汉字，未能覆盖繁体中文字、罕用字
- **1981年~1998年**：共发布了7种辅助集以完善补充
- **GB18030-2000**：《信息技术·信息交换用汉字编码字符基本集的扩充》，涵盖27484个繁简汉字
- **GB18030-2005**：《信息技术·中文编码字符集》，共收录汉字70244个，采用单字节、双字节、四字节混合编码，总编码超过150万个

GB2312-80

1980年，《信息交换用汉字编码字符集·基本集》；收录汉字**6763**个

GB18030-2000

2000年，《信息技术·信息交换用汉字编码字符集基本集的补充》，收录汉字**27484**个

GB18030-2005

2005年，《信息技术·中文编码字符集》；增加了CJK扩展B的汉字；共收录汉字**70244**个



4、汉字标准

❖ 汉字国际标准

Universal
Coded
Character Set

通用字符集 (UCS)

ISO/IEC 10646

- 国际标准化组织ISO制定
- 1984~2017年
- 对世界语言的书面形式以及其他符号进行统一编码
- 编码方案: **UCS-2**用两个字节编码, **UCS-4**用4个字节编码

同步发展

统一码 (Unicode)

- 统一码联盟制定
- 1988~2019年(V12.0)
- **统一码、万国码、单一码**: 可容纳世界上所有文字和符号, 以满足跨语言、跨平台进行文本转换、处理的要求
- 编码方案: **UTF-8**、**UTF-16**、**UTF-32**





5、汉字印刷

- ❖ 汉字印刷：将汉字原稿通过印刷机械和专用油墨，批量转印到承印物的过程和技术
- ❖ 约一千年前，我国**北宋毕昇**：创造发明了胶泥活字、木活字排版等活字印刷术，中国古代四大发明之一；后采用铅字排版进行汉字印刷
- ❖ 1975年，中国在世界上首次把**精密汉字存入了计算机**
 - **中国两院院士王选**：提出了“轮廓加参数描述汉字字形的信息压缩技术”
 - 把汉字字形信息存储量大大压缩且不失真，将几千兆字节数据压缩到几兆字节存入存储器
- ❖ 1979年，我国自主研发汉字激光照相排版系统——**“华光” 激光照排系统**



5、汉字印刷

- ❖ **“华光” 激光照排系统：我国第一个计算机中文信息处理系统**
 - 引起当代世界印刷界的惊叹，被誉为中国印刷技术的再次革命
 - 由王选主持的一项伟大发明，是我国自主创新的典型代表
 - 它的产业化和应用，淘汰了我国沿用数百年的铅字印刷
 - 华光Ⅱ型、Ⅲ型、Ⅳ型“计算机激光汉字编辑排版系统”的研制过程经历了种种困难，包括国内和国外、技术和社会多方面的因素
 - 王选凭着非凡的毅力和对创新的执著，带领研发团队，克服重重困难，使中文印刷业告别了“铅与火”，大步跨进“光与电”的时代





3.6 校验码

❖ 3.6.1 校验码概述

❖ 3.6.2 奇偶校验码

❖ 3.6.3 海明校验码





3.6.1 校验码概述

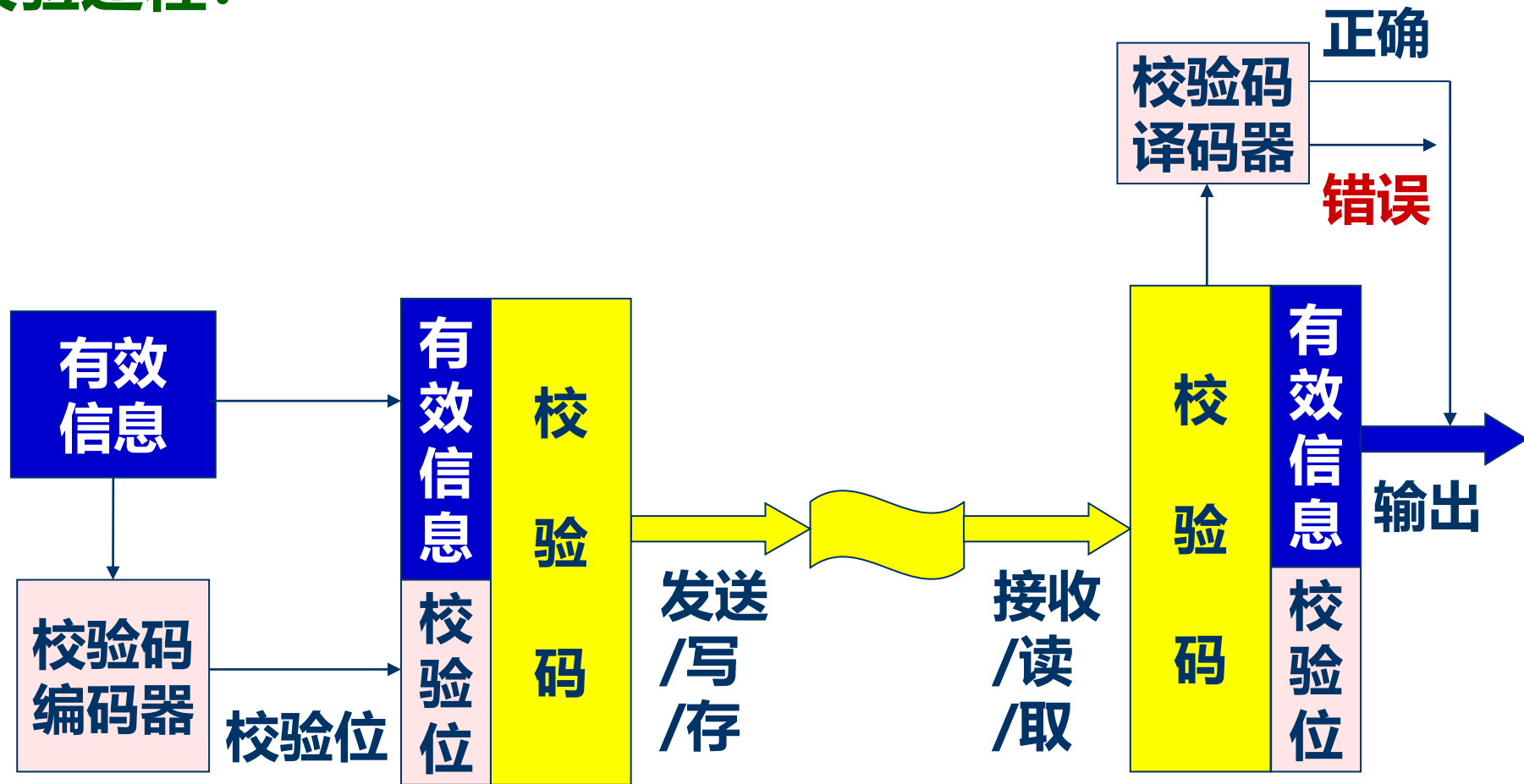
- 1、**校验码定义**：是一种具有**发现某些错误或自动改正错误能力**的一种数据编码方法
- 2、**校验码目的**：用于**检查或纠正**在存取、读写和传送数据的过程中可能出现的**错误**
- 3、**校验码的基本思想**：“**冗余校验**”，即通过在有效信息代码的基础上，添加一些冗余位来构成整个校验码
- 4、**校验码的构成**：**有效信息 + 校验位**

有效信息产生的冗余位



3.6.1 校验码概述

❖ 5、校验过程：





3.6.1 校验码概述

6、校验码原理：通过判断代码的合法性来检错的。

- 只有当合法码之间的码距 $d \geq 2$ 时，校验码才具有检错能力
- 当码距 $d \geq 3$ 时，校验码才具有纠错能力
- 码距：一种码制的码距是指该码制中所有代码之间的最小距离。
- 两个代码之间的距离：在任何两个代码之间逐位比较，对应位值不同的个数

8421码：0101和0110的距离？码距？

- 校验码的检错纠错能力与码距的关系如下：
 - ① 若码距 d 为奇数，如果只用来检查错误，则可以发现 $d - 1$ 位错误；如果用来纠正错误，则能够纠正 $(d-1) / 2$ 位错误。
 - ② 若码距 d 为偶数，则可以发现 $d/2$ 位错误，并能够纠正 $(d/2 - 1)$ 位错误。

8421码： $d=1$ ，0101变成了0100，两个都是合法码，无法检出错误

$d=1$ 为何不能检查出错误？



3.6.1 校验码概述

❖ 7、常见校验码：

校验码	码距d	检错能力	应用场合
奇偶校验码	2	检出奇数位错误	磁带、串行通信
海明校验码	≥ 3	纠错1或多位	磁盘冗余阵列
CRC校验码	3	纠错1位	磁盘、数据块的校验





3.6.2 奇偶校验码

- ❖ 在有效信息位的前面或后面 添加 1位奇（偶）校验位 → 奇（偶）校验码
- ❖ 奇（偶）校验码的编码和译码：硬件上通常采用异或非门（异或门）实现

1、编码

- 奇校验位的取值：要使整个奇校验码中 “1”的个数为奇数
- 偶校验位的取值：要使整个偶校验码中 “1”的个数为偶数
- 例：发送端，要发送七位ASCII码（B6 B5 B4 B3 B2 B1 B0）
 - 在ASCII码前面添加一位奇校验位 $P_{\text{奇}}$ 或偶校验位 $P_{\text{偶}}$ 变为一个字节的奇偶校验码，则它们的生成表达式为

$$\begin{aligned} P_{\text{奇}} &= \overline{B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0} \\ P_{\text{偶}} &= B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0 \end{aligned}$$

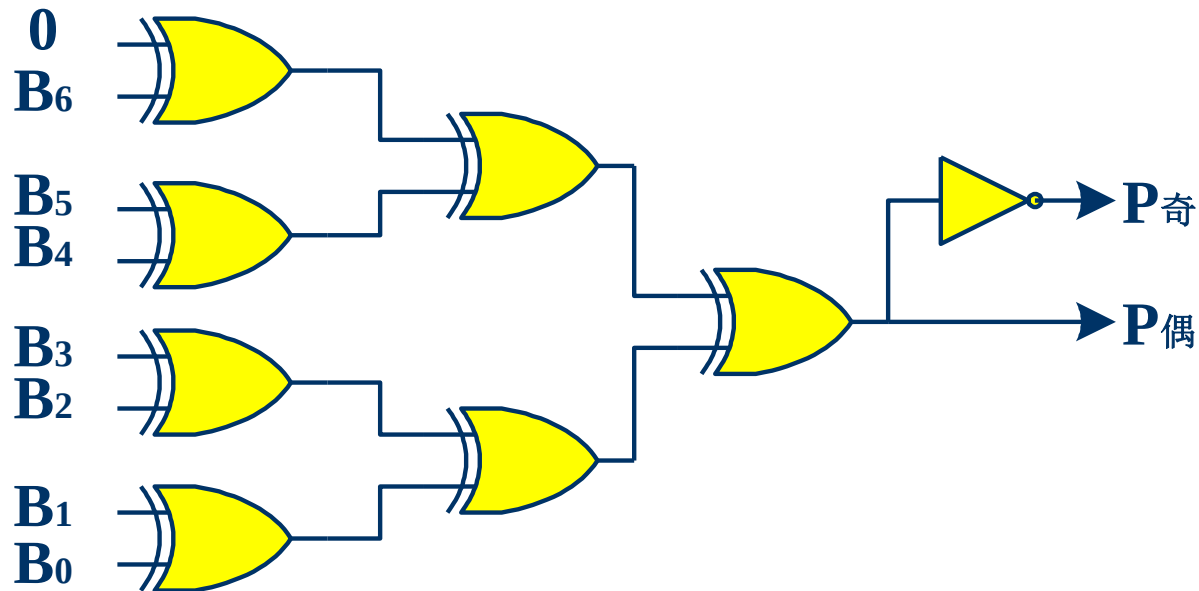


3.6.2 奇偶校验码

❖ 奇偶校验的编码电路

$$P_{\text{奇}} = \overline{B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0}$$

$$P_{\text{偶}} = B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0$$



❖ 例如：字符“A”的ASCII码为

41H = 100 0001B

❖ 奇校验码为C1H = 1100 0001

❖ 偶校验码为41H = 0100 0001



3.6.2 奇偶校验码

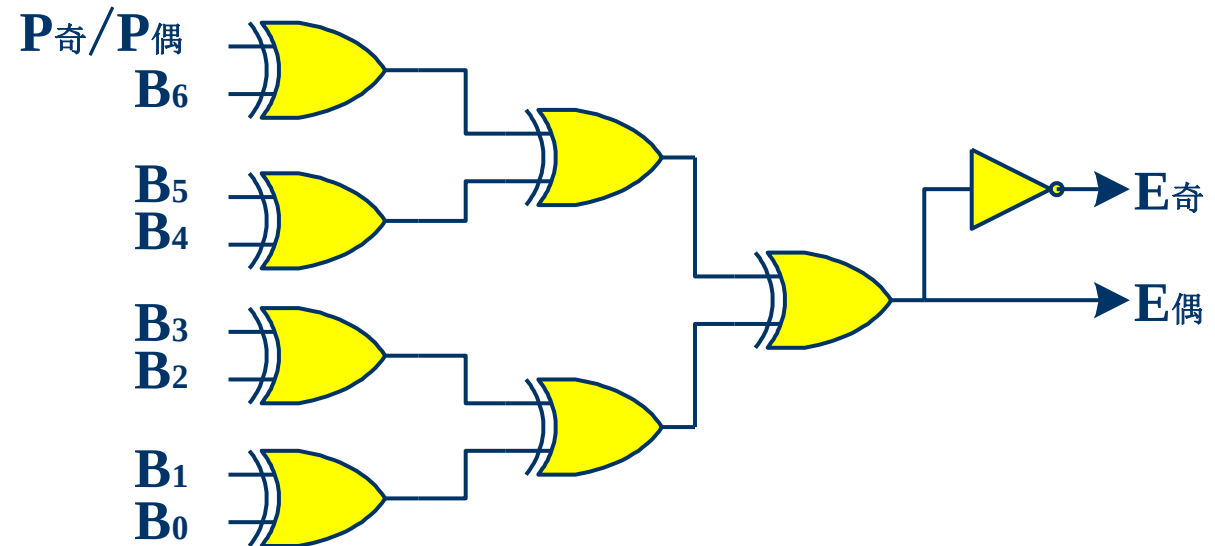
2、译码

- 接收端检验接收到的校验码信息的奇偶性，奇校验码中“1”的个数应该为奇数；偶校验码中“1”的个数应该为偶数；否则出错
- $E_{\text{奇}}$ 为奇校验码出错信号 $E_{\text{偶}}$ 为偶校验码出错信号
- 为1出错，为0正确

奇偶校验的译码表达式

$$E_{\text{奇}} = \overline{B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0 \oplus P_{\text{奇}}}$$
$$E_{\text{偶}} = B_6 \oplus B_5 \oplus B_4 \oplus B_3 \oplus B_2 \oplus B_1 \oplus B_0 \oplus P_{\text{偶}}$$

奇偶校验的译码电路





3.6.3 海明校验码

❖ 1、编码：分组奇偶校验

❖ 步骤如下：

- (1) 计算校验位的位数
- (2) 确定有效信息和校验位的位置
- (3) 分组
- (4) 进行奇偶校验，合成海明码

❖ 2、译码





(1) 计算校验位的位数

- ❖ 假设：信息位为k位，增加r位校验位，构成 $n=k+r$ 位海明码字
- ❖ 若要求海明码能纠正一位错误，用r位校验位产生的r位指误字来区分无错状态及码字中n个不同位置的一位错误状态，则要求r满足：
- ❖ $2^r \geq k + r + 1$
- ❖ 有效信息k位→能纠错一位的海明校验码的校验位的位数r

k值	r最小值
1 ~ 4	3
5 ~ 11	4
12 ~ 26	5
27 ~ 57	6
58 ~ 120	7





(2) 确定有效信息和校验位的位置

- ❖ 假设 k 位有效信息从高到低为 $D_k D_{k-1} \dots D_2 D_1$
- ❖ 添加的 r 位校验位为 $P_r P_{r-1} \dots P_2 P_1$
- ❖ 它们构成 $n=k+r$ 位的海明码为 $H_n H_{n-1} \dots H_2 H_1$
- ❖ H 下标被称为海明位号，则第 i 位的校验位 P_i 必须位于位号为 2^{i-1} 的位置，即：
- ❖ $P_i = H_j, j = 2^{i-1}$ ，其中， $i = 1, 2, \dots, r$;
- ❖ 有效信息则在其余的海明码位置上顺序排列。
- ❖ 例如： $k=8$ ，则 $r=?$ 其海明码的排列？

H_{12}	H_{11}	H_{10}	H_9	H_8	H_7	H_6	H_5	H_4	H_3	H_2	H_1
D_8	D_7	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1





(3) 分组

- ❖ 分组进行奇偶校验，每一组通过一个监督表达式来监督有效信息的变化
- ❖ 分组必须使得监督表达式得出的指误字能够反映出错位的位号。
- ❖ **分组的原则：**
 - 校验位只参加一组奇偶校验，有效信息则参加至少两组的奇偶校验。
 - 若 $D_i = H_j$ ，则 D_i 参加那些位号之和等于 j 的校验位的分组校验。

$k=8, r=4$ 的海明码分组

序号	H_{12}	H_{11}	H_{10}	H_9	H_8	H_7	H_6	H_5	H_4	H_3	H_2	H_1
分组	D_8	D_7	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1
P_4	√	√	√	√	√							
P_3	√					√	√	√	√			
P_2		√	√			√	√			√	√	
P_1		√		√		√		√		√		√





(4) 进行奇偶校验，合成海明码

- ① 按照分组和奇偶校验的规律将每个校验位的生成表达式写出
- ② 带入有效信息的值，依次得出校验位的取值
- ③ 将校验位按各自的位置插入，与有效信息一起合成海明码。

例如：有效信息为11101001，则：

可以纠错一位的海明码为1 1 1 0 1 1 0 0 0 1 0 1

$$P_4 = D_8 \oplus D_7 \oplus D_6 \oplus D_5$$

$$P_3 = D_8 \oplus D_4 \oplus D_3 \oplus D_2$$

$$P_2 = D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1$$

$$P_1 = D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1$$





2、译码

- ❖ 在接收端收到每个海明码后，也必须按上述分组检验每组的奇偶性有无发生变化；方法：
 - 按照监督关系式算出指误字 $S_r S_{r-1} \dots S_2 S_1$
 - 若为全零，则说明各组奇偶性全部无变化，信息正确，将相应的有效信息位析取出来使用；
 - 若不为零，则指误字的十进制值，就是出错位的海明位号。
- ❖ **例如：**收到的海明码为110011000101，得到 $S_4 S_3 S_2 S_1 = (1010)_2 = (10)_{10}$ ，则表明是 H_{10} (D_6) 出错，将 H_{10} 取反，得正确海明码为11**1**011000101。

$$S_4 = P_4 \oplus D_8 \oplus D_7 \oplus D_6 \oplus D_5$$

$$S_3 = P_3 \oplus D_8 \oplus D_4 \oplus D_3 \oplus D_2$$

$$S_2 = P_2 \oplus D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1$$

$$S_1 = P_1 \oplus D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1$$





3.7 现代计算机系统的数据表示

- ❖ 3.7.1 CPU硬件支持的数据类型
- ❖ 3.7.2 高级语言支持的数据类型





3.7.1 CPU硬件支持的数据类型

❖ **硬件支持的数据类型**：对存储在计算机的寄存器或者存储器中的数据，能够由指令进行哪些种类的**解释、读写和运算处理**

❖ **(1) 数据访问的尺寸**：指CPU的指令能访问的**数据位数**

❖ **编址方式**：存储器中分配地址编码的最小单位

- **按字节编址**：即每8位分配一个地址编码（大部分CPU）
- **按字编址**：每个字分配一个地址编码

1MB的存储器
需要几位地址？

❖ **MIPS、RISC-V：按字节编址**

- **32位CPU访问尺寸**：
 - 字数据（32b）
 - 半字（16b）
 - 字节数据（8b）

字长32位，
1MB的存储器
需要几位地址？

❖ **Intel x86：按字节编址**

- **访问尺寸**：
 - 字节(Byte, 8位)
 - 字(Word, 16位)
 - 双字(Dword, 32位)
 - 四字(Qword, 64位)



3.7.1 CPU硬件支持的数据类型

- ❖ **硬件支持的数据类型**：对存储在计算机的寄存器或者存储器中的数据，能够由指令进行哪些种类的解释、读写和运算处理
- ❖ **(1) 数据访问的尺寸**：指CPU的指令能访问的数据位数
- ❖ **编址方式**：存储器中分配地址编码的最小单位
 - **按字节编址**：即每8位分配一个地址编码（大部分CPU）
 - **按字编址**：每个字分配一个地址编码

❖ MIPS、RISC-V：按字节编址

- **32位CPU访问尺寸**：
 - 字数据（32b）
 - 半字（16b）
 - 字节数据（8b）

❖ 如何区分内存某个单元的数据尺寸？

- **取决于访问它的指令的操作数尺寸**
- **RISC-V**：
 - lw指令：访问32位的字数据
 - lh指令：访问16位的半字数据
 - lb指令：访问字节数据



3.7.1 CPU硬件支持的数据类型

❖ 按字节编址的存储器+能访问多种尺寸数据的CPU，如何安排存放顺序？

❖ 多字节数据：在内存中的存放顺序称为字节顺序，有两种

■ 小端模式存储：little-endian

- 低字节 (LSB) 存放在低地址
- 高字节 (MSB) 存放在高地址

■ 大端模式存储：big-endian

- 低字节 (LSB) 存放在高地址
- 高字节 (MSB) 存放在低地址

LSB? Least Significant Byte

MSB? Most Significant Byte

❖ x86 CPU：小端模式

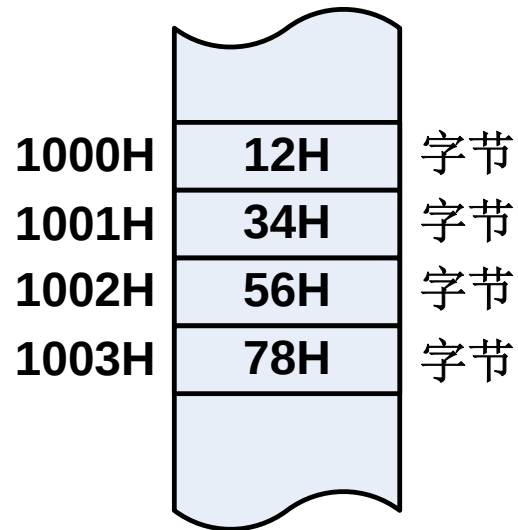
❖ MIPS：大端模式

❖ RISC-V：两种均可



3.7.1 CPU硬件支持的数据类型

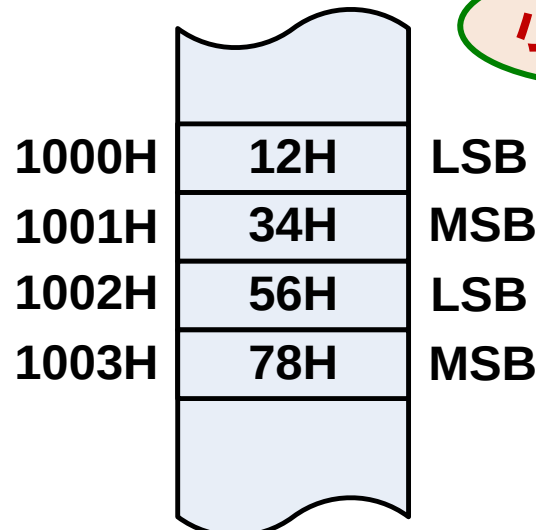
❖ 例3.11 x86微机的内存1000H~1003H单元中，分别存放了12H、34H、56H和78H这4个字节数据，写出下列x86汇编指令执行的结果。



字节数据

MOV AL,[1000H]

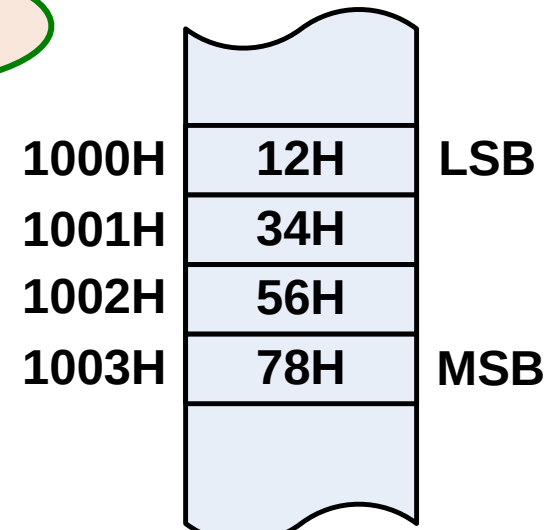
AL=12H



字数据

MOV AX,[1000H]

AX= 3412H



双字数据

MOV EAX,[1000H]

EAX= 78563412H



3.7.1 CPU硬件支持的数据类型

- ❖ (2) 数据类型：按照数据本身代表的含义划分的类别
- ❖ 某个二进制数串是什么数据类型，在机器语言级，取决于访问它的指令
- ❖ ①整数：分为有符号整数和无符号整数，有符号整数用补码表示
 - 字节数据：90H
 - 由有符号数指令访问：真值为-112
 - 由无符号数指令访问：真值是144
- ❖ ②浮点数：IEEE 754标准的单精度、双精度浮点数
 - x86的FPU也实现与支持80位的扩展双精度浮点数
 - 通常由专用的浮点数指令访问与解释
- ❖ ③字符串：由字符的ASCII码（1B）或者文字的Unicode编码（2B）组成，按顺序存放在内存或寄存器中

1000H	D0H	“中”
1001H	D6H	
1002H	4FH	“O”
1003H	4BH	“K”
1004H	21H	“!”



3.7.1 CPU硬件支持的数据类型

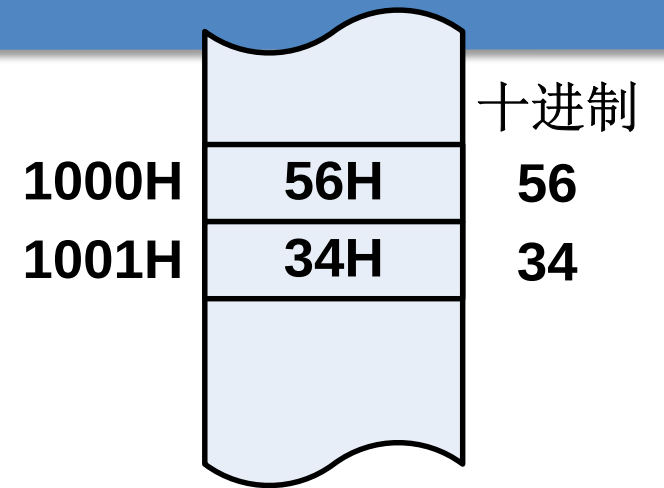
❖ (2) 数据类型

❖ ④BCD数据：大部分CPU不单独设置BCD码处理指令

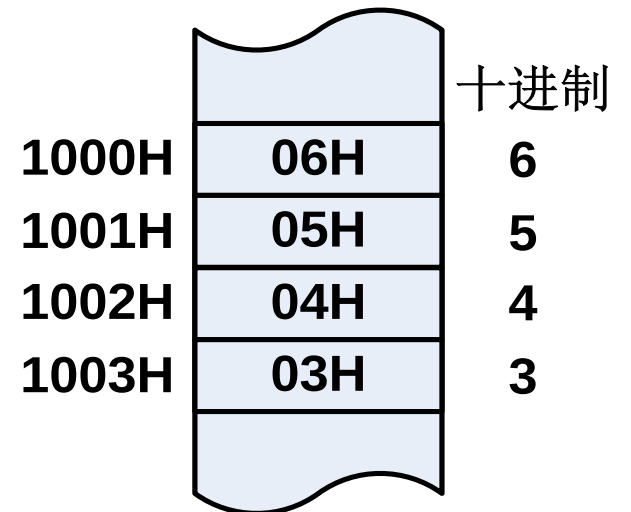
- x86 CPU：能处理
 - 压缩的BCD码(4位)
 - 非压缩的BCD码(8位)

❖ ⑤指针：主存单元的地址，无符号数据

- x86 CPU：定义了两种指针
 - 一种是32位的近指针，用于段内偏移量和段内访问
 - 另一种是48位的全指针/远指针，用于段间访问



(a) 压缩BCD码



(b) 非压缩BCD码





3.7.2 高级语言支持的数据类型

❖ 1、高级语言中的数据类型

❖ 取决于语言标准和编译器实现的目标代码；和机器字长有一定关系

❖ 32位计算机上，C语言的基本数据类型

- (1) 整型数：
 - `int/short/long/long long`
 - 32位/16位/32位/64位
 - 补码表示的有符号整数
 - 无符号数：添加`unsigned`
- (2) 浮点数：
 - `float`: 32位的短实数（单精度）
 - `double`: 64位的长实数（双精度）
- (3) 字符：8位字符ASCII码/整数
 - `char`: 8位有符号整数
 - `unsigned char`: 8位无符号整数
- (4) 指针：指针变量的值，其他变量在主存中的地址
 - 无符号数；它的尺寸与CPU的寻址能力有关
 - 对于32位CPU，指针变量一般是32位的

❖ Java语言：不同之处

- `char`: 16位的Unicode字符（无符号数）
- 有布尔类型`boolean`(8位): `true`和`false`

3.7.2 高级语言支持的数据类型

- ❖ 例3.12 下面是32位机器上的一段C语言程序，假设变量p、x和y顺序存放在主存地址1000 0800H开始的单元，主存按字节编址、小端模式。请写出程序执行结束后各变量的值。

p、x和y：什么类型的编码？
几位？带符号 or 不带符号？
原码 or 补码 or 移码？

```
int    *p;  
  
int    x=-123456;      //x=0xFFFE 1DC0 (补码)  
  
p=&x;                  //p=0x10000804  
  
int    y=*p;           //y=-123456
```

- ❖ 高级语言的整型数和浮点数都有相应的表示范围，遵循底层的二进制编码规则
- ❖ 合格的程序员：要进行逻辑严密的分析，避免出现溢出情况

地址	主存 内容	
1000 0800H	04H	p
1000 0801H	08H	
1000 0802H	00H	
1000 0803H	10H	
1000 0804H	C0H	x
1000 0805H	1DH	
1000 0806H	FEH	
1000 0807H	FFH	
1000 0808H	C0H	y
1000 0809H	1DH	
1000 080AH	FEH	
1000 080BH	FFH	
		



3.7.2 高级语言支持的数据类型

❖ 2、数据类型强制转换

- 隐式：发生在不同类型数据之间的赋值或计算时
- 显式：通过类型强制符实现

-1 < 0u: 值是false

❖ (1) 有符号数 和 无符号数之间的转换

- 多数高级语言遵循的原则：
 - 底层的二进制位保持不变
- 无符号数 和 有符号数 进行运算：
 - C语言：有符号数→无符号数，再进行计算
 - 比较运算时：容易出现意想不到的结果

❖ (2) 不同类型、不同尺寸数据之间的转换

- 在底层上：通常会改变二进制位的值
- 可能出现的问题有：
 - ①表示范围变小时：可能产生溢出
 - ②有效数字减少时：可能产生舍入，导致精度降低
 - ③整数尺寸由短变长时：
 - 无符号数要进行零扩展（高位添0）
 - 有符号数要进行符号扩展（高位添符号位）

float→int
double→float

char→short
short→int
int→long long

double→float
int→float



2、数据类型强制转换

❖ 例3.13 下列C语言语句在32位计算机上执行，写出执行后变量的十进制值和16进制编码。

语句	十进制变量值	16进制编码	备注
short si = -1234;	si=-1234	FB2E	16位补码表示-1234
int i = si;	i=-1234	FFFF FB2E	有符号数2B转换成4B，符号扩展
unsigned short usi = si ;	usi=64302	FB2E	有符号数转换成无符号数，编码不变，解释不同
unsigned int ui = usi;	ui=64302	0000 FB2E	无符号数2B转换成4B，零扩展
int y = 123456789;	y=123456789	075B CD15	32位补码表示123456789
float f1 = y;	f1=123456792	4CEB 79A3	y的有效数字超过24位，转换为float时被截断
double d = 5.8;	d=5.7999999999999998	4017 3333 3333 3333	不能精确表示
int z = (int) d;	z=5	0000 0005	将d的值向0舍入，转换为整数
float f2 = (float)d;	f2=5.80000019	40B9 999A	d的有效数字舍入后赋值给f2



2、数据类型强制转换

- ❖ 强制类型转换（尤其是隐式）：经常导致程序错误，且不易发现；常被程序员忽视，从而造成安全漏洞，甚至造成很大的损失
- ❖ 1996年6月4日：欧洲航天局发射了阿丽亚娜5型运载火箭
 - 发射后仅仅37秒，火箭就偏离飞行路径，继而解体爆炸了；损失高达6亿美元
- ❖ 控制惯性导航系统的计算机：向控制引擎喷嘴的计算机发送了一个无效数据
- ❖ 正确数据：火箭的水平速率，由64位浮点数→16位有符号数
- ❖ 阿丽亚娜4型火箭：水平速率没有超出16位数的表示范围
- ❖ 阿丽亚娜5型火箭：速度比4型火箭高出5倍，程序员却没有重新论证和测试，直接重用这一程序，转换时发生了数据溢出，导致火箭发射失败
- ❖ 数据表示中的误差、类型转换引起的错误，都有可能造成灾难性的后果
- ❖ 任何成功的项目离不开程序员和工程师严谨细致的工作





本章小结

- ❖ **数制有两个要素**：基数R与位权W；计算机中的信息均由二进制来表示，即 $R=2$ ， $W=2^i$
- ❖ **BCD码**：用4位二进制编码表示1个十进制数字
- ❖ **机器数**：数值数据在机器中的表示形式
- ❖ **根据小数点的位置是否浮动**，可以分为定点数和浮点数
- ❖ **定点机器数**：根据小数点的隐含位置又分为定点小数和定点整数两种
- ❖ **定点机器数**：有四种表示形式——原码、反码、补码和移码
- ❖ **计算机中的整型数 (int)**，是用补码表示的。
- ❖ **移码**：主要用于表示浮点数的阶码



本章小结

- ❖ **浮点机器数：由阶码E、尾数M 两部分构成**
 - 阶码E：指数，定点整数
 - 尾数M：系数，定点小数
 - 阶码E的底R：一般隐含为2
- ❖ 浮点机器数的小数点的位置随阶码数值而变化。
- ❖ **规格化的浮点数表示：尾数没有前导零。**
- ❖ **IEEE754标准的浮点数：单精度为32位，双精度为64位， $R=2$**
 - 阶码：偏移量为 2^n-1 的移码
 - 尾数：原码表示，具有隐藏位



本章小结

❖ 计算机中的非数值数据的表示：

- 西文字符：采用7位的ASCII码来表示
- 汉字的输入编码：用于使用西文标准键盘输入汉字
- 汉字的机内码：用于汉字的存储、检索和处理
- 汉字的字模码：用于汉字的显示和打印输出

❖ 校验码：用于检错和纠错

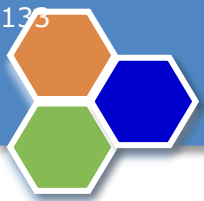
- 奇偶校验码：最简单的一种检错码，它可以检查出一位或奇数位错误
- 海明校验码：一种多重奇偶校验码，具有纠错能力



本章小结

- ❖ **计算机系统的数据类型：与CPU硬件、操作系统以及编译器有关**
 - **CPU硬件支持的数据类型：一般有整数、浮点数、字符串、BCD数据和指针等类型，有各种不同的数据访问尺寸**
 - **高级语言支持的数据类型：有整数、浮点数、字符和指针**
- ❖ **小数、浮点数的表示不精确，数据类型强制转换也会引起误差和溢出**
- ❖ **本章重点：定点机器数、浮点机器数的表示方法**





The End!

